

Noisy Function Secret Sharing and its applications to Differentially Private computations

Marc Damie^{1,2}, Federico Mazzone¹, Florian Hahn¹,
Andreas Peter³, and Jan Ramon²

¹ University of Twente, The Netherlands

² Inria, France

³ Carl von Ossietzky Universität Oldenburg, Germany

Abstract. Function Secret Sharing (FSS) schemes enable to share secret functions between multiple parties, with notable applications in anonymous communication and privacy-preserving machine learning. While two-party schemes offer logarithmic key sizes, multi-party schemes remain less practical due to significantly larger keys. Although several approaches have been proposed to improve multi-party schemes, a significant efficiency gap remains between the two-party and multi-party settings.

Our work introduces noisy FSS: a relaxation of FSS preserving the standard privacy guarantees but relaxing the correctness definition by allowing a small amount of noise in the output. We formally define noisy FSS and show how the noise introduced by the scheme can be leveraged to provide differential private outputs in statistics applications.

To demonstrate the benefits of this relaxation, we adapt a scheme proposed by Corrigan-Gibbs et al. (S&P’15). While their scheme provides the smallest key sizes among multi-party schemes, they do not support some applications notably in statistics due to their non-linear share decoding. On the contrary, recent works such as Goel et al. (CRYPTO’25) have larger keys, but support all FSS applications. Our noisy adapted scheme offers the best of both worlds by matching the best key sizes, while providing the properties necessary to statistics applications.

Keywords: Function Secret Sharing · Distributed Point Function · Learning with Error · Differential Privacy.

1 Introduction

Function Secret Sharing (FSS) [10] is a cryptographic primitive enabling to share a secret function efficiently. In particular, there has been intensive research efforts on “Distributed Point Functions” (DPF); schemes supporting point functions (a function equal to β on a point α , and 0 otherwise). FSS schemes are composed of three algorithms (Gen, Eval, Decode). A dealer knowing a secret function f generates p keys $(k_1, \dots, k_p) \leftarrow \text{Gen}(f, 1^\lambda)$, each key being sent to a different

party. For any x in the function domain, each keyholder can compute a share $\llbracket f(x) \rrbracket_j \leftarrow \text{Eval}(k_j, x)$. Like in classic secret sharing, they decode the secret $f(x) \leftarrow \text{Decode}(\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p)$.

Such schemes have applications in private information retrieval (PIR) [28], secure databases [22, 48], anonymous communications [17, 27, 34, 46], access control [38], multi-party computations (MPC) [45, 47] and private data analysis [7]. Moreover, the literature presented schemes supporting other function families such as Distributed Comparison Functions (DCF); which have applications notably in privacy-preserving Machine Learning (ML) [37], private statistics [4], and MPC [12].

State of the art The efficiency of FSS schemes is commonly evaluated based on their key size. More specifically, existing works study the key size in function of the function domain size N . For two-party and three-party DPF, there exist highly efficient schemes with logarithmic key size [10, 11, 50]. However, for arbitrary multi-party schemes, there is still active research to reach such performances.

Over the last ten years, several directions have emerged to build multi-party schemes outperforming the PRG-based scheme proposed by Boyle et al. [10], whose key size is $O(\sqrt{N}q^p)$ (for p parties and an output in $\mathbb{F}_q$). The exponential factor q^p makes this scheme impractical for any medium-sized modulus q . This impracticality has become a recurrent motivation in many works [20, 21, 29, 32]; including ours.

The first direction has been to build elliptic-curve-based schemes [17, 32]. These schemes have the best asymptotic key size $O(\sqrt{N} \log q)$ with small constant factors. However, the secrets are encoded using elliptic curve points, so they require a non-linear share decoding limiting the applications.

Second, Bunn et al. [14] have leveraged the honest-majority assumption to build more efficient schemes. They proposed two schemes: a PRG-based scheme with $O(\sqrt[4]{N}q^p)$ key size built upon [10], and an information-theoretic scheme with $O(\sqrt{N} \binom{p}{t} \log q)$ key size. As the PRG-based scheme inherits the exponential factor from [10], only the information-theoretic scheme is practical.

Third, some works [18, 23] used homomorphic secret sharing (HSS) to build FSS schemes. HSS is tightly related to FSS as it focuses on generating secret shares on which parties can evaluate functions without interactions between them. These schemes have significant constant factors in the key size, that limit their practicality. Existing FSS works [17, 14, 32, 29] usually cite them, but do not compare to them.

Finally, a work (concurrent to ours) [29] recently relaxed the privacy definition via a new definition called “ ϵ -privacy” (which has no link with differential privacy). This relaxation enables them to modify the scheme of [10] to obtain a key size of $O(\sqrt{N}p^3 \log q)$. Due to the factor p^3 , this scheme does not outperform the elliptic-curve-based schemes [17]. However, it has a linear share decoding so it supports more applications.

To sum up, all these approaches have managed to provide asymptotic and/or concrete performance improvements, but there remains a significant gap to

obtain key sizes comparable to two- and three-party schemes. This literature overview also shows that there is no single approach providing the best schemes. Ten years after the introduction of FSS, the two most noticeable multi-party schemes are the elliptic-curve-based DPF of [17] and the DPF with relaxed privacy of [29]. The former provides the best key size ($O(\sqrt{N})$ for [17] vs. $O(\sqrt{N}p^3)$ for [29]), but has limited applications due to its non-linear decoding.

Applications to statistics DPF has been invented initially to build PIR protocols [28]. While DPF has found applications in various domains, recent works have proven its benefits for private statistics protocols such as histograms [7], statistical testing [4], and sparse vector aggregation [3].

In all these applications, a set of n data owners wants to compute an aggregate statistics on data distributed among them. The FSS-based protocols to compute these statistics have the following structure: (1) Each data owner i shares (with p servers) a secret function f_i representing their data, (2) Each server j uses its FSS keys to evaluate the shares $\llbracket f_i(x) \rrbracket_j$ for each point x and each shared function f_i , (3) Each server j aggregates its shares locally $\llbracket h(x) \rrbracket_j = \sum_i \llbracket f_i(x) \rrbracket_j$, (4) The servers decode the final output $h(x)$ for each x .

While such applications are simple, they cannot be supported by elliptic-curve-based schemes due to their non-linear decoding. Indeed, the FSS correctness definition requires that $\text{Decode}(\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p) = f(x)$. However, statistics applications as described above require aggregating shares locally; i.e., we need an additional property: $\text{Decode}(\sum_i \llbracket f_i(x) \rrbracket_1, \dots, \sum_i \llbracket f_i(x) \rrbracket_p) = \sum_i f_i(x)$. Unfortunately, the non-linear decoding does not provide this property, contrary to the linear share decoding used in [10, 29].

Works in FSS [17, 32] have rarely discussed consequences of this non-linear decoding. On the contrary, we argue that linear decoding is essential for applications that have emerged in privacy-preserving statistics. This observation puts a new light on the literature: even though elliptic-curve-based schemes provide the best key size, they are limited to specific applications (contrary to schemes like [29]).

Desirable share properties This perspective based on statistics applications emphasizes an important gap: we should compare FSS schemes on more factors than just the key size. In their seminal work, Boyle et al. [10] had introduced *informally* three share properties that would be “*desirable*” to support as many FSS applications as possible. Unfortunately, no work has ever reused these terms.

A first property is “compressibility”: the ability to combine shares “in a meaningful way.” This concept directly relates to the property necessary to statistics applications, share aggregation being one method to combine shares. This property enables to pinpoint the shortcoming of elliptic-curve-based schemes: their evaluation shares are not compressible.

Second, they coined the notion of “function-private output.” While the standard FSS privacy definition prevents information leakage from the FSS keys, the function private output prevents leakage from the evaluation shares themselves. For example, let us consider a point function f for which x_0 is not the

non-zero point (i.e., $f(x_0) = 0$). The set of shares $\{\llbracket f(x_0) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p\}$ should not reveal information about the non-zero point $f(\alpha) = \beta$. From the adversary perspective, α should be (with equal probability) any point other than x_0 . The absence of this property has an important impact in practice because it requires a trusted party to decode the share as in [32], otherwise the decoding could leak information.

Finally, Boyle et al. [10] proposed the concept of “succinctness:” the evaluation shares $\llbracket f(x) \rrbracket$ should have a size “comparable” to the function output $f(x)$.

Even though they are essential to analyze the benefits and drawbacks of schemes such as [29, 32], recent works have not reused these properties introduced 10 years ago. An element limiting their use is surely the lack of formalism: Boyle et al. [10] have only given informal definitions which prevents formal proof or analysis of these additional properties.

Our contributions To build better schemes, we propose a novel approach: relaxing the correctness definition. While Goel et al. [29] relaxed the privacy definition to optimize [10], we instead allow some noise in the output. Specifically, we introduce the notion of ψ -correctness, a new FSS correctness definition based on stochastic dominance, which bounds the output noise using a distribution ψ .

We call this relaxed variant: noisy FSS. We formalize the definitions, study its properties, and demonstrate its benefits by adapting the DPF from [17] to the noisy setting. The resulting scheme combines the small key size of [17] ($O(\sqrt{N} \log q)$) with the linear decoding of [29]; offering a wide application range. In short, the relaxation scheme leads to a multi-party scheme with the best key size and functionalities.

Finally, we show how to leverage the FSS noise to provide Differential Privacy (DP) guarantees in statistics applications. Thus, noisy FSS provides *by design* stronger privacy guarantees than classic FSS. Indeed, the classic FSS privacy property guarantees that the function remains private during the MPC protocol, but the protocol output (e.g., a histogram) can still leak information about the input data. Thanks to the noise of our scheme, the leakage is inherently mitigated, and this privacy gain is quantified using DP.

Concretely, our work contains four main contributions:

1. **Introduce noisy Function Secret Sharing** by relaxing the standard FSS correctness definition.
2. Demonstrate the benefits of noisy FSS by **adapting the DPF of [17] to the noisy setting**; transforming an efficient scheme incompatible with statistics applications into a noisy scheme compatible with them.
3. Detail how to leverage the **FSS noise to provide differential privacy guarantees** in FSS applications to privacy-preserving statistics.
4. Formalize the three FSS share properties informally introduced in [10]: succinctness, function-private output, and compressibility.

2 Preliminaries

2.1 Notations

Let N be the size of the function domain, p be the number of parties, t the number of dishonest parties, 1^λ a security parameter, $\llbracket y \rrbracket_i$ the i -th share of a secret y , and $s \xleftarrow{R} S$ refer to the uniform random sampling of s from the set S . The term error distribution refers to any probability distribution with zero mean. We use ψ and χ for arbitrary error distributions.

Threat model Similarly to most FSS works [10, 11, 14, 17, 28], our work focuses on adversaries composed of semi-honest agents (following the protocol and *passively* inferring private information).

The literature distinguishes two types of scheme. On the one hand, honest-majority schemes are secure only if the majority of the agents is honest ($\forall t$ such that $t < p/2$). On the other hand, dishonest-majority schemes are secure even when there is a majority of dishonest agents ($\forall t$ such that $t < p$).

2.2 Function Secret Sharing (FSS)

FSS schemes enable to share (among p parties) secret functions from a function family $\mathcal{F}$. Each scheme is designed for a specific function family.

Function families Boyle [9] formalized the notion of function family as follows: a function family $\mathcal{F}$ is a pair $(P_{\mathcal{F}}, E_{\mathcal{F}})$ where $P_{\mathcal{F}} \subseteq \{0, 1\}^*$ is an infinite collection of function descriptions $\hat{f}$, and $E_{\mathcal{F}}: P_{\mathcal{F}} \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ is a polynomial-time algorithm defining the function described by $\hat{f}$. In other words, each function description $\hat{f} \in P_{\mathcal{F}}$ describes a corresponding function $f: f(x) = E_{\mathcal{F}}(\hat{f}, x)$. All functions belonging to the same function family have the same domain $\mathcal{X}$ and output space $\mathcal{Y}$.

There exist schemes for various function families including point functions [10], intervals [11], and decision trees [11]. Like most FSS works [10, 11, 9, 14, 17, 29, 32], we focus essentially on Distributed Point Functions (DPF) [28]; i.e., FSS schemes for point functions (functions f such that $f(x) = \beta$ if $x = \alpha$, and 0 otherwise). Note that Appendix D also presents a noisy scheme for comparison functions.

Algorithms The p -party FSS schemes have 3 algorithms:

- Gen : $P_{\mathcal{F}} \times \mathbb{N} \rightarrow \mathcal{K}^p$: takes as input a function description $\hat{f} \in P_{\mathcal{F}}$ and a security parameter $1^\lambda \in \mathbb{N}$ and outputs p keys $k_1, \dots, k_p \in \mathcal{K}$. *This algorithm is executed by the secret holder.*
- Eval : $\mathcal{K} \times \mathcal{X} \rightarrow \mathbb{G}$: takes as input a key $k_i \in \mathcal{K}$ and a point $x \in \mathcal{X}$ and outputs a value $y_i \in \mathbb{G}$. *This algorithm is executed by each shareholder.*
- Decode : $\mathbb{G}^p \rightarrow \mathcal{Y}$: takes as input p shares $\{y_1 \dots y_p\}$ and outputs $f(x)$.

Gen is a probabilistic algorithm and Decode a deterministic algorithm. While in all existing schemes Eval is deterministic, our noisy schemes have a probabilistic Eval. The decoding algorithm is not systematically included in the scheme definition by existing works [10], but we include it to have self-contained definitions (e.g., the correctness definition relies on it).

Security definitions Boyle et al. [10] introduced two security definitions for FSS schemes: correctness (of the output) and privacy (of the input function).

Definition 1 (Correctness). *Let $\mathcal{E} = (\text{Gen}, \text{Eval}, \text{Decode})$ be an FSS scheme for the function family $\mathcal{F}$. For any $f \in \mathcal{F}$ (and its corresponding description $\hat{f} \in P_{\mathcal{F}}$), for any point $x \in \mathcal{X}$, we have $k_1, \dots, k_p \leftarrow \text{Gen}(1^\lambda, \hat{f})$ and $\Pr[\text{Decode}(\text{Eval}(k_1, x), \dots, \text{Eval}(k_p, x)) - f(x) = 0] = 1$*

The initial privacy definition given by [10] was based on an indistinguishability game. More recently, Boyle [9] proposed a simulation-based alternative. In this paper, we adopt a slightly modified version of that simulation-based definition (see Definition 2). This modification simplifies the extension developed in Section 3.2, where we formalize the notion of function-private output.

Definition 2 (Privacy). *Let $\mathcal{F}$ be a set of functions with a common domain $\mathcal{X}$. An FSS scheme $\mathcal{E} = (\text{Gen}, \text{Eval}, \text{Decode})$ for $\mathcal{F}$ is private if the generation algorithm is simulatable. More formally, given a semi-honest adversary who corrupts a subset of the parties $A \subset \{1, \dots, p\}$, there exists a PPTA $\mathcal{S}$ such that for all PPTA $\mathcal{D}$, there exists a negligible function negl , such that for all functions $f \in \mathcal{F}$ (and their corresponding descriptions $\hat{f} \in P_{\mathcal{F}}$), and for all security parameters $\lambda \in \mathbb{N}$, we execute the generation algorithm $(k_1, \dots, k_p) \leftarrow \text{Gen}(1^\lambda, \hat{f})$ and require that*

$$|\Pr[\mathcal{D}(\{k_i\}_{i \in A}, f) = 1] - \Pr[\mathcal{D}(\mathcal{S}(\lambda, \mathcal{F}), f) = 1]| \leq \text{negl}(\lambda) .$$

2.3 FSS Applications to Privacy-Preserving Statistics

While FSS has been initially designed for private information retrieval [28], several works have described over the years applications to statistics. This subsection surveys these applications, and identify their common requirements.

Histograms Histograms are a basic statistical method to represent data distributions. These statistics are popular in many research fields, and can sometimes be applied on sensitive data; e.g., to represent a body weight distribution in healthcare or a salary distribution in sociology.

Private histograms are a straightforward application of DPF mentioned in some existing works [7, 24]. Algorithm 1 describes a basic DPF-based histogram-making protocol. Moreover, DPF can also implement other complex operations related to histograms such as heavy hitters [7].

Algorithm 1 Simple DPF-based histogram-making protocol

```

1: Let  $\mathcal{E} = (\text{Gen}, \text{Eval}, \text{Decode})$  be a  $p$ -party DPF scheme.
2: Let  $\{P_1, \dots, P_n\}$  be a set of  $n$  data owners, each owning a value  $e_i \in \{1 \dots N\}$ . Let  $\{S_1 \dots S_p\}$  be a set of  $p$  servers.

3: // Secret value sharing (Client side)
4: for each data owner  $P_i \in \{P_1, \dots, P_n\}$  do
5:   Define  $f_i$  a point function s.t.  $f_i(e_i) = 1$ , 0 otherwise.
6:   Generate the DPF keys  $\{k_{i,1}, \dots, k_{i,p}\} \leftarrow \text{Gen}(\hat{f}_i, \lambda)$ .
7:   Send one key  $k_{i,j}$  to each server  $S_j$ .

8: // Share aggregation (Server side)
9: for each server  $S_j \in \{S_1, \dots, S_p\}$  do
10:  Initialize the vector share  $\llbracket h \rrbracket_j \in \mathbb{G}^N$  with zeros.
11:  for each key  $k_{i,j} \in \{k_{1,j}, \dots, k_{n,j}\}$  do
12:    for each  $x \in \{1, \dots, N\}$  do
13:      Evaluate the share  $\llbracket f_i(x) \rrbracket_j \leftarrow \text{Eval}(k_{i,j}, x)$ .
14:      Update  $\llbracket h[x] \rrbracket_j \leftarrow \llbracket h[x] \rrbracket_j + \llbracket f_i(x) \rrbracket_j$ .
15:  Send the vector share  $\llbracket h \rrbracket_j$  to the other servers.

// Result decoding (Server side)
16: Decode the histogram  $h \leftarrow \text{Decode}(\llbracket h \rrbracket_1, \dots, \llbracket h \rrbracket_p)$ .

```

Empirical cumulative distribution function (ECDF) ECDF is an estimation of the cumulative distribution function of a random variable. They are an essential building block in many statistics and ML computations.

For example, Barczewski et al. [4] used them to compute ROC curves (to assess the accuracy of a classifier) or to perform a Hosmer-Lemeshow statistical test (to calibrate medical instruments). Their work proposed to compute private ECDF more efficiently via DCF.

Sparse vector aggregation Asi et al. [3] have proposed a sparse vector aggregation based on DPF. A sparse vector is a vector containing a minority k of non-zero values. Thus, such vectors can be represented as the aggregation of k point functions (one for each non-zero element in the vector). To build an FSS-based sparse vector aggregation, we can then use DPF (like in private histograms).

Secure sparse aggregations are particularly useful in Federated Learning; a decentralized ML paradigm in which parties exchange gradients to jointly train common ML model. To enhance privacy in Federated Learning, many secure aggregation protocols have been proposed [6, 33]. Over the years, several works [1, 40] have proposed to sparsify the gradients to reduce the communication costs (e.g., transmitting only the k model parameters with the highest amplitude).

Common requirements All these statistics applications follow the same structure as Algorithm 1: (1) secret sharing, (2) value/share aggregation, (3) result decod-

ing. The second step is particularly interesting because it requires aggregating shares locally (see Line 14).

This local aggregation induces two requirements. First, the FSS schemes must be efficient with output in a generic field $\mathbb{F}_m$ (for $m \gg 2$) because they need to aggregate private values up to a value m . This requirement was not present in historical FSS applications like PIR for which bit fields $\mathbb{F}_2$ were sufficient. Second, it requires a share homomorphism:

$$\begin{aligned} & \text{Decode}(\sum_i \llbracket f_i(x) \rrbracket_1, \dots, \sum_i \llbracket f_i(x) \rrbracket_p) \\ &= \text{Decode}(\llbracket \sum_i f_i(x) \rrbracket_1, \dots, \llbracket \sum_i f_i(x) \rrbracket_p) = \sum_i f_i(x) \end{aligned}$$

While these requirements seem trivial several multiparty schemes do not satisfy them. On the one hand, PRG-based multi-party schemes [10, 14] are exponential in the field size. For example, Boyle et al. [10] have a factor q^p in their key size. Thus, they are impractical in generic fields. On the other hand, elliptic-curve-based schemes [17, 32] do not provide the share homomorphism due to their non-linear decoding [32]. Indeed, such schemes commonly encode secret values $\beta \in \mathbb{F}_q$ using an elliptic curve point $P_\beta \in E(\mathbb{F}_q)$ such that $\text{xCoordinate}(P_\beta) = \beta$. This creates a non-linear decoding, and breaks the homomorphism (because, in the general case, $\text{xCoordinate}(P_\beta + P_\gamma) \neq \beta + \gamma$).

3 Formalizing the desirable share properties

Boyle et al. [10] have mentioned some desirable share properties: **compressibility, function-private output, and succinctness**. These properties were only informally presented, because their presence was trivially proven thanks to the linear secret sharing used in [10].

However, in noisy FSS (defined in Section 4), their presence is not straightforward (notably function-private output). Thus, this section formalizes these share properties, so we can formally prove them for our scheme.

3.1 Succinctness

This first “desirable” property was presented by [10] as follows: a share output by the Eval algorithm is “succinct” if it is “comparable in size to the function output”. This property should not be confused with “compact key sizes:” while FSS works usually want to minimize key size (the Gen output), succinctness refers to the size of the function output shares (the Eval output). For example, sharing a function truth table is a trivial FSS implementation. This trivial scheme has FSS keys as large as the function domain, but the evaluation shares are succinct because they have (individually) the same size as the function output.

We propose to formalize succinctness as follows:

Definition 3 (Δ -succinctness). A scheme has Δ -succinct shares if $\log_2(\#\mathbb{G}) \leq \log_2(\#\mathcal{Y}) + \Delta$ with $\mathbb{G}$ the “share space” and $\mathcal{Y}$ the function output space.

The value Δ is the bit-length difference between a function output share and the function output $f(x)$. Existing schemes [10, 11, 14, 17] provide 0-succinctness.

Definition 3 gives an “additive” formulation to this notion of succinctness. Other formulations based on multiplicative relationships or polynomial dependencies could be possible. However, we prefer this additive formulation because it makes the added cost visible (i.e., the bit-size difference). Moreover, these additional bits have a precise role in noisy FSS: they store the noise. Thus, this additive definition resonates with Fully Homomorphic Encryption schemes [16] in which low-order bits are also dedicated to noise handling.

Note that any scheme (even poorly efficient) can satisfy Δ -succinctness for a sufficiently large Δ . A scheme is considered succinct if it satisfies it for a “sufficiently” small Δ . For instance, Garbled Circuits can be used to construct a simple two-party FSS scheme [10]: given a function f , the first key consists of the garbled circuit representing x , while the second key contains the information needed to map each input x to the corresponding garbled input labels. However, this scheme is not considered succinct by [10] because its output shares have a size of $O(L)$ for a function with a circuit size of L .

Remark 1 (Quantized functions). Certain FSS applications involve real values (e.g., privacy-preserving ML [32, 37]), but the use of cryptographic tools in such protocols commonly require quantizing these values. Our definitions consider $\mathcal{Y}$ to be the output space of the quantized function.

3.2 Function-Private Output (FPO)

While the FSS privacy definition (Definition 2) prevents information leakage from the keys, the shares obtained using Eval can also reveal unintended information. During FSS-based protocols (like Algorithm 1), parties sometimes exchange evaluation shares, but these shares may leak information. To prevent this leakage, Boyle et al. [10] introduced the term *function-private output*. Informally, a scheme provides a *function-private output* if, given the evaluation shares $\{\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p\}$, an adversary learns no more information than the value $f(x)$.

To formalize this concept, we extend Definition 2. The idea is to secretly evaluate a function f at an arbitrary number of points $x_1, \dots, x_n$ in the function domain $\mathcal{X}$. The scheme has function-private output if the evaluation shares for these points can be simulated using only the $x_1, \dots, x_n$, their evaluations $f(x_1), \dots, f(x_n)$, the function family $\mathcal{F}$, and λ .

This expresses the intuition that the evaluation shares reveal no additional information about f beyond what can already be inferred from its evaluations on the chosen points $f(x_1), \dots, f(x_n)$, and the general structure of $\mathcal{F}$. In particular, they do not reveal any information about $f(x)$ for any $x \notin \{x_1, \dots, x_n\}$. It is

important that the simulator has access to the function family $\mathcal{F}$: for example, if $\mathcal{F}$ is the class of point functions and $\alpha \in \{x_1, \dots, x_n\}$, then it is trivial to deduce that $f(x) = 0$ for all the remaining elements x of the domain.

Definition 4 (Function-Private Output). *Let $\mathcal{F}$ be a set of functions with a common domain $\mathcal{X}$. An FSS scheme $\mathcal{E} = (\text{Gen}, \text{Eval}, \text{Decode})$ for $\mathcal{F}$ has function-private output (FPO) if the evaluation shares of any subset of $\mathcal{X}$ are simulatable.*

More formally, given a semi-honest adversary who corrupts a subset of the parties $A \subset \{1, \dots, p\}$, there exists a PPTA $\mathcal{S}$ such that for all PPTA $\mathcal{D}$, there exists a negligible function negl , such that for all functions $f \in \mathcal{F}$ (and their corresponding descriptions $\hat{f} \in P_{\mathcal{F}}$), for all subsets $X = \{x_1, \dots, x_n\} \subset \mathcal{X}$, and for all security parameters $\lambda \in \mathbb{N}$, we execute the generation algorithm $(k_1, \dots, k_p) \leftarrow \text{Gen}(1^\lambda, f, p)$, and the evaluation and decoding for each point in $x_j \in X$, namely

$$\begin{aligned} y_{i,j} &\leftarrow \text{Eval}(k_i, x_j) \quad \forall i \in \{1, \dots, p\} \\ y_j &\leftarrow \text{Decode}(y_{1,j}, \dots, y_{p,j}) \end{aligned}$$

and require that

$$\begin{aligned} &|\Pr[\mathcal{D}((\text{realView}, \text{realOut}), f, \{x_1, \dots, x_n\}) = 1] \\ &\quad - \Pr[\mathcal{D}((\text{simView}, \text{idealOut}), f, \{x_1, \dots, x_n\}) = 1]| \\ &\leq \text{negl}(\lambda) \end{aligned}$$

where

$$\begin{aligned} \text{realView} &:= \{k_i\}_{i \in A}, \{y_{i,j}\}_{i \in A, j \in \{1, \dots, n\}}, \\ &\quad \{(x_1, y_1), \dots, (x_n, y_n)\} \\ \text{realOut} &:= \{(x_1, y_1), \dots, (x_n, y_n)\} \\ \text{simView} &:= \mathcal{S}(\lambda, \mathcal{F}, \{(x_1, f(x_1)), \dots, (x_n, f(x_n))\}) \\ \text{idealOut} &:= \{(x_1, f(x_1)), \dots, (x_n, f(x_n))\} . \end{aligned}$$

When $n = 0$, Definition 4 corresponds to Definition 2. Indeed, the classic privacy definition of Definition 2 only prevents the leakage at key generation time, while the function-private output of Definition 4 also prevents leakage during the function evaluation. Thus, function-private output can be interpreted as an “ultimate” form of privacy in FSS.

Function-private output is important in practice because FSS applications such as in privacy-preserving ML [32] require revealing specific points of the function. However, the adversary should not discover more than the expected points. Recently, Kumar et al. [32] introduced an FSS scheme requiring a *trusted decoder* in order to guarantee the function privacy. Other existing schemes verify naturally this concept, notably thanks to the use of linear secret sharing [10].

3.3 Compressibility

Finally, Boyle et al. [10] mentioned a last share property: compressibility. They defined it as the possibility to combine shares “in a meaningful way.” This informal definition captures many possible scenarios from the simple aggregation of multiple shares to complicated non-linear functions of the shares. However, this informality complicates the analysis and comparison of FSS schemes.

We propose to specify this compressibility definition. Our formal definition focuses on “linear compressibility” because all existing works required summing function output shares. We can then formalize this property as follows:

Definition 5 (Linear Compressibility). *Let $\mathcal{E} = (\text{Gen}, \text{Eval}, \text{Decode})$ be an FSS scheme for the function family $\mathcal{F}$. The scheme $\mathcal{E}$ provides linear compressibility if for any number of functions $f_1, \dots, f_m \in \mathcal{F}$ (and their corresponding descriptions $\hat{f}_1, \dots, \hat{f}_m \in P_{\mathcal{F}}$), corresponding number of elements $x^{(1)}, \dots, x^{(m)} \in \mathcal{X}$, and $\lambda \in \mathbb{N}$, the following condition holds*

$$\Pr \left[\text{Decode} \left(\sum_{j=1}^m \text{Eval}(k_1^{(j)}, x^{(j)}), \dots, \sum_{j=1}^m \text{Eval}(k_p^{(j)}, x^{(j)}) \right) = \sum_{j=1}^m f_j(x^{(j)}) \right] = 1$$

where $(k_1^{(j)}, \dots, k_p^{(j)}) \leftarrow \text{Gen}(\hat{f}_j, 1^\lambda)$ for all $j \in \{1, \dots, m\}$.

Informally, if $\llbracket f(x) \rrbracket_i = \text{Eval}(k_i^{(f)}, x)$ is a share of $f(x)$ and $\llbracket g(x') \rrbracket_i = \text{Eval}(k_i^{(g)}, x')$, we have $\llbracket f(x) \rrbracket_i + \llbracket g(x') \rrbracket_i = \llbracket f(x) + g(x') \rrbracket_i$ (i.e., a share of $f(x) + g(x')$).

Moreover, we can comment a few details from Definition 5. First, this definition assumes that the functions $f_1, \dots, f_m$ come from the same function family $\mathcal{F}$. This definition could be generalized to multiple function families (possibly with different domains) *if the FSS schemes use the same decoding algorithms*. For example, with additive secret shares, we can trivially aggregate additive shares $y_i^{(f)}$ from a point function f with additive shares $y_i^{(g)}$ from a comparison function g . Second, two functions $f_j, f_{j'}$ from $f_1, \dots, f_m$ can be equal. Such equality would cover applications in which we want to aggregate points from the same function. For example, DPF-based PIR [28] usually evaluates a function on its whole function domain and then aggregate all the shares.

Schemes based on linear secret sharing naturally satisfy this property [10]. However, elliptic-curve-based schemes do not have such homomorphism because they encode the secret values using elliptic curve points: for a secret β , the point P_β has β as x -coordinate and the shares of P_β are p points $\{P_1 \dots P_p\}$ such that $\sum_i P_i = P_\beta$. If we have two shared secrets β and γ , in the general case, $P_\beta + P_\gamma \neq P_{\beta+\gamma}$; i.e., no compressibility. The absence of compressibility makes these schemes unsuitable for many applications such as PIR or privacy-preserving statistics.

4 Noisy Function Secret Sharing

4.1 Core noisy definitions

Noisy FSS schemes have exactly the same algorithms as classic FSS schemes: Gen, Eval, Decode. The privacy definition remains also unchanged: the adversary should not be able to extract information from t out of p keys. The only difference is the correctness definition: Decode outputs $f(x) + \text{err}$ (with err a noise) instead of $f(x)$.

Noisy definitions in cryptography Noisy FSS is not the first noisy cryptographic primitive [8, 49]. For instance, Boneh et al. [8] introduced the notion of “almost” seed-homomorphic Pseudo-Random Generators (PRG):

Definition 6. A PRG G is γ -almost seed-homomorphic if

$$G(s_1) + G(s_2) = G(s_1 + s_2) + \text{err} \text{ with } \text{err} \in [-\gamma, \gamma] \text{ a small bounded error.}$$

Towards general definitions Fixing bounds like in [8] provides an easily understandable definition, but it is limiting. For example, Gaussian noises are unbounded. For our definitions, we want to adopt a more generic approach.

To build our noisy correctness definition, we then rely on “stochastic dominance” [30, 39], a partial order between random variables. Let $\succsim$ refer to this partial order:

Definition 7 (Stochastic dominance). Given two random variables A, B , we say that A has first-order stochastic dominance over B if

$$\Pr[A \geq t] \geq \Pr[B \geq t] \quad \forall t$$

and we denote it by $A \succsim B$ or equivalently $B \preccurlyeq A$.

To better understand this stochastic order $\succsim$, we can observe a consequence of stochastic dominance: $A \succsim B \Rightarrow \mathbb{E}[f(A)] \geq \mathbb{E}[f(B)]$, for all monotone function f . In particular, if f is the identity function, $A \succsim B \Rightarrow \mathbb{E}[A] \geq \mathbb{E}[B]$; i.e., A takes in average higher values than B .

Based on this order, we propose the following “noisy correctness” definition:

Definition 8 (ψ -noisy correctness). Let $\mathcal{E} = (\text{Gen}, \text{Eval}, \text{Decode})$ be an FSS scheme for the function family $\mathcal{F}$, with common domain $\mathcal{X}$. For any function $f \in \mathcal{F}$ (and its corresponding description $\hat{f} \in P_{\mathcal{F}}$), $x \in \mathcal{X}$, and $\lambda \in \mathbb{N}$, we define the random variable

$$\mathcal{D}_{f,x}^\lambda := \text{Decode}(\text{Eval}(k_1, x), \dots, \text{Eval}(k_p, x))$$

with $(k_1, \dots, k_p) \leftarrow \text{Gen}(\hat{f}, 1^\lambda)$.

Let ψ be an error distribution. We say that $\mathcal{E}$ satisfies ψ -noisy correctness (or simply is ψ -noisy) if for all $f \in \mathcal{F}$, $x \in \mathcal{X}$, and $\lambda \in \mathbb{N}$:

$$(\mathcal{D}_{f,x}^\lambda - f(x))^2 \preccurlyeq \psi^2.$$

Thanks to the square exponent, this definition bounds *symmetrically* the distribution of $\mathcal{D}_{f,x}^\lambda$ based on an error distribution ψ . We could rephrase this definition as follows: the distribution $\mathcal{D}_{f,x}^\lambda$ is more “concentrated” (i.e., less noisy) than the error distribution ψ .

The stochastic dominance used in Definition 8 provides a more holistic definition than fixed bounds because we bound the error based on the error distribution ψ . On the one hand, if the error distribution is naturally bounded, we can naively consider these bounds for our error term. On the other hand, if the error distribution is unbounded (or loosely bounded), we can deduce bounds holding with high probability. For example, a Gaussian noise is unbounded, but we can set bounds that a Gaussian noise will not exceed (with high probability).

Moreover, we can consider the influence of the number of parties, p , on the noise distribution ψ . Ideally, one may like to make this dependence explicit; such as by including a linear factor in p directly in the Definition 8. However, we use a more generic formulation to ensure compatibility with a broad range of noisy schemes. While the noise distribution is typically influenced by p , the relationship is not necessarily linear. For instance, in our construction (described in Section 5), the noise variance includes a factor of $(p+1)$. Defining ψ generically provides greater flexibility, enabling researchers to develop schemes that rely on a wide variety of (potentially unforeseen) noise distributions.

Remark 2 (Fixed distribution). An FSS scheme $\mathcal{E}$ for the function family $\mathcal{F}$ (with domain $\mathcal{X}$) is ψ -noisy correct if

$$(\mathcal{D}_{f,x}^\lambda - f(x)) \sim \psi \quad (\forall f \in \mathcal{F}, \forall x \in \mathcal{X}).$$

This remark provides an intuitive understanding of noisy correctness: the decoding output follows a reference probability distribution. While this version is easier to understand, it is not as general as Definition 8. Definition 8 only bounds the noise distribution, so the noise distribution can be “smaller” than ψ on some specific points $x \in \mathcal{X}$ or functions $f \in \mathcal{F}$; which is the case in our schemes. Remark 2 is more rigid and imposes the noise distribution to be exactly the same for all functions and points.

For any arbitrary algorithm, we could find a random variable ψ such that the algorithm satisfies Definition 8. This “tautological” satisfaction was also present in the γ -almost seed-homomorphic PRGs [8] as their correctness definition could be satisfied by any algorithm if γ is large. However, the goal of such noisy definitions is to be satisfied for “useful” noise distributions. In our case, we can notably deduce a remark for Gaussian error distributions:

Remark 3 (Gaussian noise). An FSS scheme $\mathcal{E}$ for the function family $\mathcal{F}$ (with domain $\mathcal{X}$) is ψ -noisy with $\psi \sim \mathcal{N}(0, \sigma^2)$ if

$$\Pr[(\mathcal{D}_{f,x}^\lambda - f(x))^2 \geq t] \leq 2 \cdot \Phi\left(\frac{\sqrt{t}}{\sigma}\right) - 1, \quad \forall t \geq 0$$

where Φ is the Gaussian cumulative distribution function.

Remark 2 leads us to another interesting observation:

Remark 4. “Classic” FSS schemes satisfy ψ -noisy FSS schemes for any error distribution ψ .

Indeed, FSS schemes satisfying Definition 1 also satisfy Definition 8 because their corresponding variable “ $(\mathcal{D}_{f,x}^\lambda - f(x))^2$ ” follows a Dirac distribution on 0. In other words, noisy correctness generalizes the correctness definition commonly used in existing works.

Noisy secret sharing Based on our noisy FSS definitions, we can naturally define the notion of noisy secret sharing. However, we have not identified any specific applications or efficiency gain provided by noisy secret sharing, contrary to noisy FSS for which our paper presents schemes improving the state of the art.

Talwar [44] has proposed “differential” secret shares enabling to share secret real numbers based on differential privacy noise. Even though Talwar uses noise in their secret sharing scheme, our noisy definitions take a completely orthogonal approach. Talwar’s “differential” shares rely on a relaxed privacy definition compared to traditional secret shares (and maintain the correctness definition). On the contrary, our noisy secret sharing relax the correctness definition and keep the same privacy definition as classic FSS.

To frame our difference with [44], we must look at the motivations behind each construction. On the one hand, differential secret sharing generalizes the notion of secret sharing to real values. On the other hand, noisy secret sharing tolerates noise to scale existing (function) secret sharing schemes. Like classic secret sharing schemes, our definitions rely on finite fields. Thus, noisy FSS is a new concept in the literature.

4.2 Noisy share properties

Noisy FSS schemes do not require relaxing the succinctness definition. However, we need to generalize the notion of function-private output (given in Definition 4). In the “classic” FPO definition, the ideal functionality is deterministic: it returns exact evaluations of the function f at the points $x_1, \dots, x_n$. This deterministic functionality is incompatible with noisy FSS, so we transform it into a probabilistic functionality: it now outputs random variables whose distributions are centered around the expected value $f(x) + e$, where $e \sim \psi$ for some error distribution ψ :

Definition 9 (ψ -Noisy Function-Private Output). *Let $\mathcal{F}$ be a set of functions with a common domain $\mathcal{X}$ and $\psi = \{\psi_n\}_{n \in \mathbb{N}}$ be a probability ensemble, where each ψ_n has zero mean. An FSS scheme $\mathcal{E} = (\text{Gen}, \text{Eval}, \text{Decode})$ for $\mathcal{F}$ has ψ -noisy FPO if the evaluation shares of any subset of $\mathcal{X}$ are simulatable, up to an error ψ_λ (depending on the security parameter λ).*

The formalization is the same as in Definition 4, but now

$$\text{idealOut} := \{(x_1, f(x_1) + e_1), \dots, (x_n, f(x_n) + e_n)\}$$

with $e_1, \dots, e_n \sim \psi_\lambda$.

Remark 5 (Definition 4 $\Rightarrow$ Definition 9). The deterministic FPO definition is an instantiation of the noisy FPO definition for $\psi \sim \delta_0$, the Dirac distribution on 0.

While classic FSS schemes based on linear secret sharing inherently satisfy the FPO property [10], noisy schemes do not necessarily offer the same guarantee. If the noise distribution is correlated with the secret function, the output may leak information. Section 5 shows how to use “noise flooding” [19] to guarantee FPO in our scheme adapted from [17].

Finally, the noisy correctness requires relaxing the linear compressibility. To do so, we can combine noisy correctness (Definition 8) with the linear compressibility (Definition 5):

Definition 10 (ψ_* -noisy linear compressibility). Let $\mathcal{E} = (\text{Gen}, \text{Eval}, \text{Decode})$ be an FSS scheme for the function family $\mathcal{F}$, with common domain $\mathcal{X}$. For $f_1, \dots, f_m \in \mathcal{F}$ (and their corresponding descriptions $\hat{f} \in P_{\mathcal{F}}$), $x^{(1)}, \dots, x^{(m)} \in \mathcal{X}$, and $\lambda \in \mathbb{N}$, we define the random variable

$$\mathcal{D}_{f_1, x^{(1)}, \dots, f_m, x^{(m)}}^\lambda := \text{Decode} \left(\sum_{j=1}^m \text{Eval}(k_1^{(j)}, x^{(j)}), \dots, \sum_{j=1}^m \text{Eval}(k_p^{(j)}, x^{(j)}) \right)$$

where $(k_1^{(j)}, \dots, k_p^{(j)}) \leftarrow \text{Gen}(\hat{f}_j, 1^\lambda)$ for all $j \in \{1, \dots, m\}$.

Let ψ be an error distribution over $\mathcal{X}$. We say that $\mathcal{E}$ provides ψ_* -noisy linear compressibility if for any number of functions $f_1, \dots, f_m \in \mathcal{F}$, corresponding number of elements $x^{(1)}, \dots, x^{(m)} \in \mathcal{X}$, and $\lambda \in \mathbb{N}$, the following condition holds

$$\left(\mathcal{D}_{f_1, x^{(1)}, \dots, f_m, x^{(m)}}^\lambda - \sum_{j=1}^m f_j(x^{(j)}) \right)^2 \preceq \psi_*^2.$$

We can also prove that noisy correctness implies noisy linear compressibility if a scheme has a linear share decoding:

Theorem 1. (Linear decoding + noisy correctness $\Rightarrow$ noisy compressibility)

Let $\mathcal{E} = (\text{Gen}, \text{Eval}, \text{Decode})$ be an FSS scheme for the function family $\mathcal{F}$.

If $\text{Decode}(y_1, \dots, y_p) = \sum_{i=1}^p y_i$ and $\mathcal{E}$ is ψ -noisy correct, then $\mathcal{E}$ provides ψ_* -noisy linear compressibility,

with ψ_* s.t. $\Pr[\psi_*^2 \geq t] = \min\{1, m \Pr[\psi^2 \geq t/m^2]\}, \forall t$.

Proof. The proof is given in Appendix A.

Theorem 1 illustrates that the noise amplitude naturally increases when we aggregate multiple noisy shares. The theorem provides a first bound for the noise distribution induced by the aggregation of multiple noisy function shares. Tighter bounds could be possible by taking into account the property of ψ (for

a given ψ -correct scheme). We leave the analysis of tighter bounds for future work.

Similarly to our noisy correctness definition, we use a generic noise distribution ψ_* in Definition 10, without explicitly specifying the influence of p and m . As in the case of noisy correctness, these parameters naturally affect the noise, but their impact is not necessarily linear or immediately apparent. Theorem 1 illustrates this well, presenting a non-linear relationship. Therefore, we once again choose a generic form for ψ_* to preserve flexibility for potential future schemes.

4.3 Handling the noise in statistics applications

This relaxed definition of FSS naturally raises the question: how to handle the noise in FSS applications? We identify two main strategies for handling the noise: mitigating its impact through a *scale-and-truncate* approach, or leveraging it as a source of *differential privacy*.

Scale-and-truncate Noise truncation/rounding is a straightforward solution: we scale the secret function such that $f(x) \gg \text{err}$, so the noise can be truncated from the final result. In other words, we reserve the low-order bits to the error (which is classic in applications of noisy cryptographic primitives [8]).

While this approach is unusable on historical FSS applications such as PIR (because the noise order would be proportional to the database size), it is compatible with the applications to statistics. Such applications usually require the aggregation of n functions $f_1, \dots, f_n$, so we would encode these functions such that $f_i(x) \gg n \cdot \text{err}$.

This approach has one main advantage: the output will not be noised (i.e., same correctness as classic FSS). However, the values must be scaled significantly to ensure that the noise remains much smaller than the secrets. This scaling will degrade the evaluation share succinctness because the bit size must be increased to enable the scaling.

Differential privacy (DP) The second option is to keep the noise and use it to provide DP guarantees. The FSS privacy definitions (Definitions 2 and 4) only protect the information leakage from the Eval and Gen outputs. However, the output of Decode can still leak information about the inputs of an FSS-based protocol (e.g., a private histogram-making). DP has emerged to mitigate such information leakage from public outputs.

By keeping the noise, we provide better privacy guarantees than classic FSS: the public output of FSS-based protocols also offers significant privacy guarantees. Such privacy guarantees are particularly valuable in statistics applications for which DP has become an essential privacy-enhancing technique [41]. Thus, Section 6 shows how to integrate the FSS noise into the DP analysis of such applications.

5 Noisy DPF scheme adapted from [17]

This section adapts the elliptic-curve-based scheme described in [17] to the noisy setting. This adaption relies on the Learning with Error (LWE) hardness assumption in order to replace the elliptic-curve-based components present in [17]. This adapted schemes provides the state-of-the-art key size of [17], while also offering the compressibility property absent from [17] but necessary to statistics applications.

5.1 LWE-based pseudo-random generator (PRG)

First, let us introduce the LWE (decision) problem [36]:

Definition 11 (Decision Learning with Errors). *Let $q \geq 2$ be a modulus, n be the dimension, m be the number of samples, and χ be an error distribution (typically Gaussian) over $\mathbb{Z}_q$. Let $A \in \mathbb{Z}_q^{n \times m}$ be a random public matrix. Let $s \in \mathbb{Z}_q^m$ be a secret vector, and $e \xleftarrow{R} \chi^m$ be an error vector.*

An adversary solves the Decision-LWE problem if they can distinguish with non-negligible probability $(A, As + e)$ from (A, u) , with u a vector uniformly sampled from $\mathbb{Z}_q^n$.

From LWE, we can build a PRG $G_{LWE} : \mathbb{Z}_q^m \rightarrow \mathbb{Z}_q^n$ s.t. $G_{LWE}(s) = As + e$, for any seed $s \in \mathbb{Z}_q^m$, and A a public matrix (randomly sampled). Note that this LWE-based PRG [8] is “almost” seed-homomorphic because $G_{LWE}(s_1) + G_{LWE}(s_2) = G_{LWE}(s_1 + s_2) + \text{err}$, with err an error term.

For simplicity, our scheme relies solely on the basic LWE-based PRG G_{LWE} , but other almost seed-homomorphic PRGs are also good foundations to build noisy FSS schemes.

5.2 Scheme

To build a first noisy DPF, we modify the scheme of Corrigan-Gibbs et al. [17]. As their scheme is built upon a DDH-based PRG (i.e., another form of seed-homomorphic PRG), the main change is to replace this PRG with an LWE-based PRG. We present our adapted scheme in Algorithm 2.

Note that the LWE-based PRG G_{LWE} is an essential component of our scheme. On the one hand, it provides the pseudorandomness necessary to prove the scheme privacy (Theorem 2). On the other hand, it provides the seed homomorphism necessary to the scheme correctness (Theorem 3).

To build their scheme, Corrigan-Gibbs et al. [17] relied on elliptic curves because they provide the only known *exact* seed-homomorphic PRG. The other *known* seed-homomorphic PRGs are all based on LWE (or LWE variants) [8], but they are *almost* seed-homomorphic (i.e., noisy). In [17], the authors could not consider noisy PRGs because they were no definitions or applications allowing for noise in FSS schemes. As our work formalizes noisy FSS and provides guidelines

Algorithm 2 Dishonest-majority noisy DPF scheme

-
- 1: Let $\nu \leftarrow \lceil \sqrt{N} \rceil$
 - 2: Represent the function domain $\{1 \dots N\}$ as a $(\nu \times \nu)$ grid.
 - 3: Let $G_{LWE} : \mathbb{Z}_q^m \rightarrow \mathbb{Z}_q^\nu$ be an LWE-based PRG using the Gaussian distribution $\mathcal{N}(0, \sigma_{LWE}^2)$ as error distribution.
 $\triangleright$ NB: $m = O(\lambda)$
 - 4: **function** Gen($\lambda, (\alpha, \beta), p$)
 - 5: Parse α as (α_l, α_r) (i.e., position in the $\nu \times \nu$ grid)
 - 6: Sample random vectors $b_1, \dots, b_{p-1}$ from $(\mathbb{Z}_q)^\nu$
 - 7: Sample vectors of seeds $s_1, \dots, s_{p-1}$ from $(\mathbb{Z}_q^m)^\nu$
 - 8: Sample a single seed s_* from $\mathbb{Z}_q^m$
 - 9: Set $b_p \in (\mathbb{Z}_q)^\nu$ such that $\sum_{i=1}^p b_i = e_{\alpha_l}$
 $\triangleright e_x$ is a vector equal to 1 on index x and 0 otherwise
 - 10: Set $s_p \in (\mathbb{Z}_q^m)^\nu$ such that $\sum_{i=1}^p s_i = s_* \cdot e_{\alpha_l}$
 - 11: Let $v \leftarrow \beta \cdot e_{\alpha_r} - G_{LWE}(s_*)$
 - 12: Build the DPF keys $k_1, \dots, k_p$ as follows:
 $k_i = (b_i, s_i, v)$
 - 13: **return** $(k_1, \dots, k_p)$
 - 14: **function** Eval $_\chi(k_i, x)$
 - 15: Parse k_i as (b_i, s_i, v)
 - 16: Parse x as (x_l, x_r) (i.e., position in the $\nu \times \nu$ grid)
 - 17: Expand the x_l -th seed: $g_i \leftarrow G_{LWE}(s_i[x_l])$
 - 18: Let $y_i \leftarrow g_i[x_r] + b_i[x_l]v[x_r]$
 - 19: Sample an error $\text{err} \in \mathbb{Z}_q$ using the distribution χ
 $\triangleright$ The error err is sampled using fresh randomness.
 - 20: Let $y_i \leftarrow y_i + \text{err}$ $\triangleright$ To have a function-private output
 - 21: **return** y_i
 - 22: **function** Decode($y_1, \dots, y_p$)
 - 23: **return** $\sum_{i=1}^p y_i$
-

for its use in concrete applications, the adaptation of [17] to the noisy setting is now possible.

Besides the LWE-based PRG, our scheme has another key difference compared to [17]: Line 20 of Algorithm 2. At the end of the Eval $_\chi$, the evaluator adds a noise drawn from the distribution χ . This noise is sampled using fresh randomness at each evaluation (i.e., not deterministically from a PRG with a fixed seed contained in the key). Thus, the evaluation has two noise sources: the LWE-based PRG, and the χ noise.

At this stage, this additional noise could seem artificial and useless. Indeed, one may think that the LWE-based PRG would be sufficient to satisfy all privacy properties, but this is untrue. The LWE noise is sufficient to satisfy the standard FSS privacy. However, the supplementary χ noise is necessary to also satisfy function-private output (see Theorem 4).

Our key size is $O(\sqrt{N}(2 + \lambda) \log q)$. Algorithm 2 represents the function domain $\{1, \dots, N\}$ as a $\sqrt{N} \times \sqrt{N}$ grid, but we can slightly optimize the key size with an asymmetric grid $N_1 \times N_2$, with $N_1 = \sqrt{N}(1 + \lambda)^{-1/2}$ and $N_2 = \sqrt{N}(1 + \lambda)^{1/2}$. The resulting key size is $\sqrt{N(1 + \lambda)} 2 \log q$. Boyle et al. [10] used a similar approach to optimize their key size.

Setting the LWE parameter Our key size is proportional to the security parameter λ . However, this parameter is not directly visible in the LWE definition (Definition 11). LWE security depends on n, m, q, χ . The parameters n and q are fixed by the application: n is the size of the PRG output ($\lceil \sqrt{N} \rceil$ in our case), and $\mathbb{Z}_q$ is the function output space.

We must tune m and χ to ensure security. However, we commonly pick the error distribution χ minimizing the error amplitude, so the noise can be handled by the downstream applications. In our work, we use Gaussian distributions as error distributions, with σ_{LWE}^2 , their variance.

Thus, our security is conditioned by m : the larger m , the harder LWE is. The parameter m is then correlated to λ .

To find exact values for the LWE parameters, we refer to existing parameter selection tools [2, 5].

5.3 Standard security properties

Theorem 2 (Privacy). *Let $\lambda \in \mathbb{N}$, $N, p \in \mathbb{N}$. For any noise distribution χ , the scheme $\mathcal{E} = (\text{Gen}, \text{Eval}_\chi, \text{Decode})$ as described in Algorithm 2 is an FSS scheme for the family of all point functions with $\alpha \in \{1, \dots, N\}$ and $\beta \in \mathbb{Z}_q$.*

Assuming that the LWE assumption holds, then the scheme $\mathcal{E}$ is private against at most t semi-honest parties with $t < p$.

Proof. As our noisy DPF is a variant of the DPF proposed by Corrigan-Gibbs et al. [17], we adapt their proof easily.

To prove the privacy (Definition 2), we must show that (for every set of t corrupted parties) there exists an efficient simulator that, for any input functions, outputs samples from a distribution that is computationally indistinguishable from the distribution of the DPF keys.

The simulator Sim returns the following: $\tilde{b}_i \xleftarrow{R} (\mathbb{Z}_q)^\nu$, $\tilde{s}_i \xleftarrow{R} \mathbb{Z}_q^m$, $\tilde{v} \xleftarrow{R} \mathbb{Z}_q^\nu$.

Since the vectors b_i and s_i vectors outputted by Gen are also random, the simulator's output distributions $\tilde{b}_i$ and $\tilde{s}_i$ are computationally indistinguishable from that induced by the distribution of a single output of Gen.

By assumption, G_{LWE} is a PRG because LWE is hard. The v vector output by Gen is computationally indistinguishable from random, since it is masked with the output of the PRG G_{LWE} seeded with a seed unknown to the adversary (i.e., s_*). As highlighted by Corrigan-Gibbs et al. [17], an efficient algorithm to distinguish v from random can also distinguish the PRG output from random.

Remark 6. The privacy of the noisy DPF described in Algorithm 2 holds for any distribution χ used in the Eval_χ ; including the Dirac distribution on 0 (i.e., no added noise).

While the privacy hold for any χ , Subsection 5.4 shows that the function-private output only holds for some distributions.

Theorem 3 (Correctness). *Let $\mathcal{E} = (\text{Gen}, \text{Eval}_\chi, \text{Decode})$ be the DPF scheme described in Algorithm 2, with $\chi \sim \mathcal{N}(0, \sigma_{\text{flood}}^2)$. Let the noise used for the LWE-based PRG and for flooding have Gaussian distributions with standard deviation σ_{LWE} and σ_{flood} , respectively. Then $\mathcal{E}$ is ψ -noisy correct with $\psi \sim \mathcal{N}(0, (p+1)\sigma_{\text{LWE}}^2 + p\sigma_{\text{flood}}^2)$.*

Proof. The proof is given in Appendix B.

This theorem highlights a quite intuitive results: the scheme is ψ -noisy with ψ being the convolution of the different noise distributions added during the generation and evaluation algorithms. As we use Gaussian distributions, we are particularly interested in the variance of $\text{Var}(\psi) = (p+1)\sigma_{\text{LWE}}^2 + p\sigma_{\text{flood}}^2$.

Let us dissect this variance. Remember that this variance corresponds to the variance of the decoded output $\text{Decode}(\text{Eval}(k_1, x), \dots, \text{Eval}(k_p, x))$. Observe that each evaluation $\text{Eval}(k_i, x)$ induces an LWE noise and a flooding noise. These evaluations are then responsible of $p(\sigma_{\text{LWE}}^2 + \sigma_{\text{flood}}^2)$ (i.e., $\sigma_{\text{LWE}}^2 + \sigma_{\text{flood}}^2$ per evaluation). After considering these evaluation-induced noises, there remains an extra unexplained σ_{LWE}^2 in $\text{Var}(\psi)$. This last term comes from the LWE-based PRG evaluation performed during the generation. Indeed, the vector v contained in each FSS key is masked using an LWE-based PRG. This “noisy masked vector” v is then responsible for this part of the scheme noise. For a detailed analysis, we refer to the proof in Appendix B.

5.4 Share properties

Succinctness As the decoding is linear, we can share any secret function with output in $\mathbb{Z}_q$ with 0-succinct shares as the shares outputted by Eval are in $\mathbb{Z}_q$.

However, we may reserve the low-order to the noise (and scale the secret accordingly). Thus, our scheme is theoretically 0-succinct, but practitioners may pick slightly larger prime fields in order to accommodate the noise. The secret scaling based on the noise is dependent on the application (and the expected accuracy), so we refer to Section 6 that explains how to leverage the FSS noise in DP computations.

Function-private output As mentioned before, while the addition of χ might seem arbitrary and useless, it is mandatory to preserve function-private output. To understand it, let us consider Algorithm 2 without the χ noise addition.

Let us analyze the protocol output for a point function f (with $f(\alpha) = \beta$) and an evaluation of $f(x)$ with $x \neq \alpha$. Without this χ noise, $\text{Decode}(\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p)$ would include p LWE noises if $x_l \neq \alpha_l$; i.e., $\text{Decode}(\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p) = p \cdot \text{err}_{\text{LWE}}$. On the contrary, if $x_l = \alpha_l$, we would have $\text{Decode}(\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p) = (p+1) \cdot \text{err}_{\text{LWE}}$. The extra err_{LWE} in the second case come from the vector v because it was masked during Gen using the noisy PRG G_{LWE} . The proof of Theorem 3 further details the phenomenon.

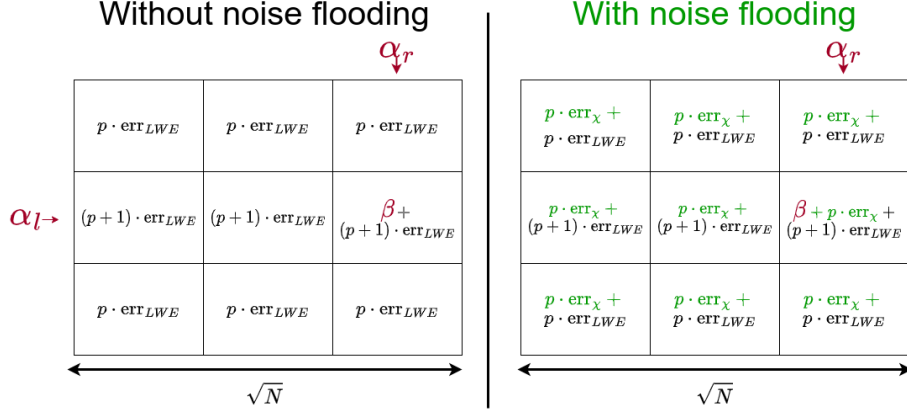


Fig. 1. Output of $\text{Decode}(\llbracket f(x) \rrbracket_1, \dots, \llbracket f(x) \rrbracket_p)$ (for all x in the domain represented as a grid) using our noisy scheme with and without noise flooding.

Using this difference in noise distribution, an adversary can partially learn the value α from the evaluation and decoding of $f(x)$, even if we do not evaluate $f(\alpha)$. This leakage is exactly what function-private output aims to prevent!

To avoid this leakage, Eval_χ adds a noise term $\text{err}_{\text{flood}} \leftarrow \chi$ to mask the LWE noise, a technique known as noise flooding [19], also used in Fully Homomorphic Encryption. This χ noise (sampled on Line 20 of Algorithm 2) “floods” the LWE noise and ensure that the noise distribution is the same on all points (from a PPT adversary perspective). Figure 1 sketches this solution. However, function privacy is only preserved if χ is chosen such that a PPT adversary cannot distinguish $\text{err}_{\text{flood}} + \text{err}_{\text{LWE}}$ from $\text{err}_{\text{flood}}$ alone. We formalize this requirement in the following theorem, which specifies the necessary conditions on χ to ensure FPO in our scheme:

Theorem 4 (Function-private output). *Let $\mathcal{E}$ be the FSS scheme described in Algorithm 2. Let the noise used for the PRG and for flooding have Gaussian distribution with standard deviation σ_{LWE} and σ_{flood} , respectively. If $\sigma_{\text{flood}} \geq \sqrt{2^{\lambda-1.5}/(p-t) - 1} \sigma_{\text{LWE}}$, with t being the number of corrupted parties, then $\mathcal{E}$ has ψ -noisy FPO with $\psi \sim \mathcal{N}(0, (p+1)\sigma_{\text{LWE}}^2 + p\sigma_{\text{flood}}^2)$.*

Proof. The proof is given in Appendix C.

Compressibility As our scheme provides noisy correctness and has a linear share decoding, according to Theorem 1, our scheme also provides noisy linear compressibility.

5.5 Comparison with state-of-the-art

Table 1 compares our noisy DPF to existing schemes, based on their key size and share compressibility. We also include the “trivial scheme”: sharing the secret function truth table.

Based on key sizes, we can categorize existing schemes in three categories. First, the PRG-based schemes with an exponential factor [10, 14], that makes them impractical. Second, the elliptic-curve-based schemes [17, 32] with no dependency on the number of parties: $O(\sqrt{N}(\lambda + \log q))$ key size. Third, the schemes with a polynomial dependency on p : $O(\sqrt{N} \cdot \binom{p-1}{t} \cdot \log q)$ for the information-theoretic scheme of [14] and $O(\sqrt{N} \cdot p^3 \lambda^3 \cdot (\log q + \lambda))$ for the PRG-based of [29].

While elliptic-curve-based schemes [17, 32] have the best key size, they lack an essential property: compressibility. Thus, contrary to other schemes, they cannot support statistics applications described in Section 2.3.

Finally, our adaptation of [17] provides the best of both worlds: the **best asymptotic key size** with no dependence on p (i.e., $O(\sqrt{N} \cdot \sqrt{(1 + \lambda)} \cdot \log q)$) and **all desirable share properties**. In other words, we improved the initial scheme by providing an essential share property absent from [17].

Table 1. Comparison of the multi-party DPF schemes.

Notations: p parties, t dishonest parties, N domain size, output in $\mathbb{Z}_q$.

Scheme	Hardness	Compress- ibility	FPO	Key size
Dishonest-majority schemes				
Trivial	None	✓	✓	$N \cdot \log q$
[10]	PRG	✓	✓	$O(\sqrt{N} \cdot q^{\frac{p-1}{2}} \cdot (\log q + \lambda))$
[17]	DDH	✗	✓	$O(\sqrt{N} \cdot (\lambda + \log q))$
[32]	DDH	✗	✗	$O(\sqrt{N} \cdot (\lambda + \log q))$
[29]	PRG	✓	✓	$O(\sqrt{N} \cdot p^3 \lambda^3 \cdot (\log q + \lambda))$
Ours	LWE	✓(noisy)	✓	$O(\sqrt{N} \cdot \sqrt{(1 + \lambda)} \cdot \log q)$
Honest-majority schemes				
[14]	PRG	✓	✓	$O(\sqrt[4]{N} \cdot \sqrt{q^{p^t}} \cdot \binom{p-1}{t} \cdot (\lambda + \log q))$
[14]	None	✓	✓	$O(\sqrt{N} \cdot \binom{p-1}{t} \cdot \log q)$

6 Leveraging FSS noise in DP computations

Several works [35, 43, 42] have studied the interactions between “cryptographic noise” and DP. On the one hand, [35, 43] have shown how to leverage the noise introduced by homomorphic encryption schemes. On the other hand, Stevens

et al. [42] designed an LWE-based protocol to securely aggregate dense vectors; using the LWE noise to provide DP guarantees.

Similarly, we want to show how to leverage the noise induced by our noisy schemes for DP guarantees in statistics applications. We demonstrate this approach for private histograms and highlight promising future directions to design FSS-based differentially private computations.

Thus, noisy FSS provides *by design* stronger privacy guarantees than classic FSS. Indeed, the classic FSS privacy property guarantees that the function remain private during the MPC protocol, but the MPC output (e.g., a histogram) can still leak information about the input data. As we can leverage the FSS noise to build DP guarantees, the leakage is mitigated in statistics applications based on noisy schemes.

Note that classic FSS protocols (e.g., [29]) can be updated to add a DP noise on their shares at the end of their Eval algorithm. Such a modification would transform them directly into a noisy scheme as they would no longer satisfy the classic FSS correctness. Hence, adapting classic FSS to provide DP guarantees naturally leads to (trivial) noisy schemes.

6.1 Background: Differential Privacy (DP)

Differential Privacy (DP) [25, 41] is a statistical framework to control private information leakage in published data. This formal privacy definition has found applications in many fields from databases to machine learning.

Definition 12 ((ϵ, δ)-Differential Privacy). *A randomized algorithm $\mathcal{A} : \mathcal{X}^n \rightarrow \mathcal{Y}$ is (ϵ, δ)-differentially private (DP) if, for any datasets $X, X' \in \mathcal{X}^n$ differing of a single element, and for all $E \subseteq \mathcal{Y}$:*

$$\Pr[\mathcal{A}(X) \in E] \leq e^\epsilon \cdot \Pr[\mathcal{A}(X') \in E] + \delta$$

In (ϵ, δ)-DP, the parameter ϵ quantifies the privacy guarantees: the smaller ϵ is, the more private a computation is. The parameter δ is a small constant to accommodate highly unlikely events in order to simplify the analysis.

Definition 12 focuses on datasets X and X' differing on a single element [26]. This concept has link with a highly important concept in DP: sensitivity. Sensitivity bounds the influence of a single input on the algorithm output. Sensitivity can be expressed using different metrics, notably the l_2 norm:

Definition 13 (Sensitivity). *For any function $f : \mathcal{X}^n \rightarrow \mathcal{Y}$, the l_2 sensitivity Δ_2 is:*

$$\Delta_2 = \max_{\substack{X, X' \in \mathcal{X}^n \\ X, X' \text{ differ on one element}}} \|f(X) - f(X')\|_2$$

DP works commonly add noises to enforce a certain level of privacy [26]. There exists various DP “mechanisms” to enhance privacy. Our work is particularly interested in the Gaussian mechanism described in Proposition 1. This proposition details the relationship between the privacy level ϵ and σ^2 , when an algorithm output is noised using a Gaussian with variance σ^2 .

Proposition 1 (Gaussian Mechanism [26]). *Let $f : \mathcal{X}^n \rightarrow \mathbb{R}^d$ be an arbitrary d -dimensional function and Δ_2 be its l_2 sensitivity. Let $F : \mathcal{X}^n \rightarrow \mathbb{R}^d$ be a randomized algorithm such that $\forall x, F(x) \leftarrow f(x) + e$ with $e \xleftarrow{R} \mathcal{N}(0, \sigma^2 \times \mathbb{I}_{d \times d})$. For any $\epsilon \in (0, 1)$, if $\sigma^2 \geq \frac{2\Delta_2 \ln(1.25/\delta)}{\epsilon^2}$, F is (ϵ, δ) -DP.*

Unfortunately, the Gaussian mechanism is only compatible with classic Gaussian noises in $\mathbb{R}$. As observed in [31], private computations relying on cryptographic protocols require discretizing the noise distribution, because the operations are commonly performed in Abelian groups (e.g., $\mathbb{Z}_q$). This discretization has an important impact on the distribution properties. For example, while the sum of continuous Gaussian distributions is a continuous Gaussian, the sum of discrete Gaussian distributions is not exactly a discrete Gaussian [31].

Thus, [15, 31] redeveloped the DP results for discrete Gaussian distributions. The continuous and discrete Gaussian distributions provide close (but not equal) privacy guarantees [31]. For our work, we are particularly interested in the privacy guarantees provided by a sum of discrete Gaussians. We remind the results of Kairouz et al. [31] in Proposition 2. In this proposition, they rely on an alternative DP definition: concentrated DP (see Definition 14). This relaxed definition provided a better framework to obtain a tighter privacy bound.

Proposition 2 (“Sum of discrete Gaussians” mechanism [31]). *Let $f : \mathcal{X}^n \rightarrow \mathbb{R}^d$ be an arbitrary d -dimensional function and Δ_p be its l_p sensitivity. Let $F : \mathcal{X}^n \rightarrow \mathbb{R}^d$ be a randomized algorithm such that $\forall x, F(x) \leftarrow f(x) + \sum_{i=1}^p e_i$, with each e_i drawn independently from a discrete Gaussian $\mathcal{N}_{\mathbb{Z}}(0, \sigma^2 I_d)$.*

For any $\sigma \geq \frac{1}{2}$, F is $\frac{1}{2}\epsilon^2$ -concentrated DP for

$$\epsilon = \min \left\{ \begin{array}{l} \sqrt{\frac{\Delta_2^2}{p\sigma^2} + \frac{1}{2}\tau d}, \\ \sqrt{\frac{\Delta_2^2}{p\sigma^2} + 2\frac{\Delta_1}{\sqrt{N}\delta}\tau + \tau^2 d}, \\ \frac{\Delta_2}{\sqrt{p}\sigma} + \frac{1}{2}\tau\sqrt{d}, \end{array} \right\}$$

with $\tau = 10 \cdot \sum_{k=1}^{p-1} e^{2\pi^2\sigma^2 \frac{k}{k+1}}$

Definition 14 (Concentrated DP [13]). *A randomized algorithm $\mathcal{A} : \mathcal{X}^n \rightarrow \mathcal{Y}$ is $\frac{1}{2}\epsilon^2$ -concentrated DP if, for any datasets $X, X' \in \mathcal{X}^n$ differing of a single element, we have*

$$\sup_{\alpha \in (1, \infty)} \frac{1}{\alpha} D_{\alpha}(M(X) || M(X')) \leq \frac{1}{2}\epsilon^2$$

with D_{α} the Rényi divergence.

6.2 Differentially private histogram with FSS noise

As detailed in Algorithm 1, to compute the histogram of n data inputs $x_1, \dots, x_n \in \{1 \dots N\}$, we need to aggregate n secret point functions $f_1, \dots, f_n$ such that $f_i(x_i) = 1, 0$ otherwise. In Algorithm 1, if the DPF scheme $\mathcal{E}$ is a noisy scheme,

the protocol would output a histogram noised with a sum of discrete Gaussians. This subsection analyses this noise and the DP guarantees that it provides.

Theorem 3 provides a stochastic dominance on our DPF noise distribution. However, DP guarantees require knowing what is the *minimum* noise added to a single point. The Eval_χ of Algorithm 2 systematically adds a noise drawn from a distribution χ to the evaluation results.

As explained in Theorem 4, we use $\chi \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma_{\text{flood}}^2)$ to guarantee FPO (see the corresponding proof for a lower bound on σ_{flood}^2). Thus, each noisy function output (i.e., $\sum_{i=1}^p \text{Eval}_\chi(k_i, x) \approx f_i(x)$) integrates a noise drawn from the sum of p discrete Gaussians with variance σ_{flood}^2 .

A private histogram requires aggregating n (noisy) point functions, so the total noise distribution added to the histogram is the sum of $n \times p$ discrete Gaussians with variance σ_{flood}^2 . We can trivially apply the results from Kairouz et al. [31] (Proposition 2) for $\sigma = \sigma_{\text{flood}}$.

For basic histograms in which each data owner contributes to a single bin with a value 0 or 1, the sensitivity is $\Delta_1 = \Delta_2 = 1$. Based on this sensitivity and our discrete Gaussian variance σ_{flood}^2 , we can deduce the privacy level using Proposition 2.

We could very slightly lower the ϵ by also taking into account the LWE noise. However, the LWE noise and the flooding noise have different variances (σ_{LWE}^2 and σ_{flood}^2). Thus, we cannot apply naively Proposition 2 because it requires discrete Gaussians with identical variance; we would need to redevelop this complex bound for a sum of discrete Gaussians with different variances.

Integrating the LWE noise would not significantly improve the ϵ . As explained in Theorem 4, the flooding noise is set to hide the influence of the LWE noise (i.e., to guarantee function-private output). The amplitude of the LWE noise is multiple orders of magnitude smaller than the flooding noise. Thus, the LWE noise would have a negligible impact on ϵ .

Note that Theorem 4 provides a lower bound on the noise variance to guarantee function-private output. We can naturally increase the variance σ_{flood}^2 (above the lower bound defined in Theorem 4) to lower ϵ ; enhancing then the privacy guarantees.

6.3 Potential extensions

Our goal in this section was to highlight the essential DP results necessary to incorporate FSS noise in differentially private computations. We demonstrated these results on a simple example, but several extensions are possible.

First, we could combine the FSS noise with other DP mechanisms. Noisy FSS provides “DP for free” as the noise is inherently present in the scheme. However, this noise could be combined with other noises in the DP computation (e.g., sharing secret functions f_i such that $f_i(x_i + e) = 1$ instead of $f_i(x_i) = 1$) to have the best accuracy-privacy trade-off.

Second, we have only presented noisy schemes based on Gaussian noises, but LWE variants (e.g., Learning with Rounding [8]) can output other noise distributions. Thus, future research could also study noisy FSS schemes with

non-Gaussian noise. Such research could require new DP results compatible with these noise distributions.

Conclusion

First, our paper formalized FSS properties that had been introduced informally in existing works. Then, we proposed a relaxed form of FSS allowing for some noise in the function output: noisy FSS. We adapted an existing scheme of [17] in order to build a noisy DPF providing both the best key size, and all desirable share properties. Finally, we showed how to leverage this noise to provide differential privacy guarantees in FSS applications to statistics.

Acknowledgments. This work was supported by the Netherlands Organization for Scientific Research (NWO) in the context of the SHARE project [CS.011] and by the Horizon Europe project HE TRUMPET.

A large language model was carefully used to check grammar and polish parts of this paper.

References

1. Aji, A.F., Heafield, K.: Sparse Communication for Distributed Gradient Descent. In: Palmer, M., Hwa, R., Riedel, S. (eds.) *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. pp. 440–445. Association for Computational Linguistics, Copenhagen, Denmark (2017). <https://doi.org/10.18653/v1/D17-1045>, <https://aclanthology.org/D17-1045>
2. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of Learning with Errors. *Journal of Mathematical Cryptology* **9**(3), 169–203 (Oct 2015). <https://doi.org/10.1515/jmc-2015-0016>, <https://www.degruyterbrill.com/document/doi/10.1515/jmc-2015-0016/html>, publisher: De Gruyter Section: Journal of Mathematical Cryptology
3. Asi, H., Feldman, V., Keller, H., Rothblum, G.N., Talwar, K.: PREAMBLE: Private and Efficient Aggregation of Block Sparse Vectors and Applications (2025), <https://eprint.iacr.org/2025/490>
4. Barczewski, A., Mawass, A., Ramon, J.: Differentially Private Empirical Cumulative Distribution Functions (Feb 2025). <https://doi.org/10.48550/arXiv.2502.06651>, <http://arxiv.org/abs/2502.06651>, arXiv:2502.06651 [cs]
5. Biasioli, B., Kirshanova, E., Marcolla, C., Rovira, S.: A Tool for Fast and Secure LWE Parameter Selection: the FHE case (2024), <https://eprint.iacr.org/2024/1895>
6. Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H.B., Patel, S., Ramage, D., Segal, A., Seth, K.: Practical Secure Aggregation for Privacy-Preserving Machine Learning. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. pp. 1175–1191. ACM, Dallas Texas USA (2017). <https://doi.org/10.1145/3133956.3133982>, <https://dl.acm.org/doi/10.1145/3133956.3133982>
7. Boneh, D., Boyle, E., Corrigan-Gibbs, H., Gilboa, N., Ishai, Y.: Lightweight Techniques for Private Heavy Hitters. In: *2021 IEEE Symposium on Security and Privacy (SP)*. pp. 762–776 (2021). <https://doi.org/10.1109/SP40001.2021.00048>

8. Boneh, D., Lewi, K., Montgomery, H., Raghunathan, A.: Key Homomorphic PRFs and Their Applications. In: Canetti, R., Garay, J.A. (eds.) *Advances in Cryptology – CRYPTO 2013*. pp. 410–428. Springer, Berlin, Heidelberg (2013). https://doi.org/10.1007/978-3-642-40041-4_23
9. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y.: Function Secret Sharing and Homomorphic Secret Sharing (2022)
10. Boyle, E., Gilboa, N., Ishai, Y.: Function Secret Sharing. In: Oswald, E., Fischlin, M. (eds.) *Advances in Cryptology - EUROCRYPT 2015*. pp. 337–367. Lecture Notes in Computer Science, Springer, Berlin, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46803-6_12
11. Boyle, E., Gilboa, N., Ishai, Y.: Function Secret Sharing: Improvements and Extensions. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. pp. 1292–1303. ACM, Vienna Austria (2016). <https://doi.org/10.1145/2976749.2978429>, <https://dl.acm.org/doi/10.1145/2976749.2978429>
12. Boyle, E., Gilboa, N., Ishai, Y.: Secure Computation with Preprocessing via Function Secret Sharing. In: Hofheinz, D., Rosen, A. (eds.) *Theory of Cryptography*. pp. 341–371. Lecture Notes in Computer Science, Springer International Publishing, Cham (2019). https://doi.org/10.1007/978-3-030-36030-6_14
13. Bun, M., Steinke, T.: Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds. In: Hirt, M., Smith, A. (eds.) *Theory of Cryptography*. pp. 635–658. Springer, Berlin, Heidelberg (2016). https://doi.org/10.1007/978-3-662-53641-4_24
14. Bunn, P., Kushilevitz, E., Ostrovsky, R.: CNF-FSS and Its Applications. In: *Public-Key Cryptography – PKC 2022*, vol. 13177, pp. 283–314. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-030-97121-2_11, https://link.springer.com/10.1007/978-3-030-97121-2_11
15. Canonne, C., Kamath, G., Steinke, T.: Discrete Gaussian for Differential Privacy. *Journal of Privacy and Confidentiality* **12**(1) (Jul 2022). <https://doi.org/10.29012/jpc.784>, <https://journalprivacyconfidentiality.org/index.php/jpc/article/view/784>
16. Coron, J.S., Lepoint, T., Tibouchi, M.: Scale-Invariant Fully Homomorphic Encryption over the Integers. In: Krawczyk, H. (ed.) *Public-Key Cryptography – PKC 2014*. pp. 311–328. Springer, Berlin, Heidelberg (2014). https://doi.org/10.1007/978-3-642-54631-0_18
17. Corrigan-Gibbs, H., Boneh, D., Mazières, D.: Riposte: An Anonymous Messaging System Handling Millions of Users. In: *2015 IEEE Symposium on Security and Privacy*. pp. 321–338 (2015). <https://doi.org/10.1109/SP.2015.27>
18. Couteau, G., Kumar, N., Ye, X.: Multiparty Homomorphic Secret Sharing and More from LPN and MQ (2025), <https://eprint.iacr.org/2025/966>
19. Dahl, M., Demmler, D., El Kazdadi, S., Meyre, A., Orfila, J.B., Rotaru, D., Smart, N.P., Tap, S., Walter, M.: Noah’s Ark: Efficient Threshold-FHE Using Noise Flooding. In: *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. pp. 35–46. WAHC ’23, Association for Computing Machinery, New York, NY, USA (Nov 2023). <https://doi.org/10.1145/3605759.3625259>, <https://dl.acm.org/doi/10.1145/3605759.3625259>
20. Damie, M., Hahn, F., Peter, A., Ramon, J.: DDH-based schemes for multi-party Function Secret Sharing (2025), <https://eprint.iacr.org/2025/1805>

21. Damie, M., Hahn, F., Peter, A., Ramon, J.: Eliminating Exponential Key Growth in PRG-Based Distributed Point Functions (2025). <https://doi.org/10.48550/arXiv.2509.22022>
22. Dauterman, E., Rathee, M., Popa, R.A., Stoica, I.: Waldo: A Private Time-Series Database from Function Secret Sharing. In: 2022 IEEE Symposium on Security and Privacy (SP). pp. 2450–2468 (2022). <https://doi.org/10.1109/SP46214.2022.9833611>
23. Dodis, Y., Halevi, S., Rothblum, R.D., Wichs, D.: Spooky Encryption and Its Applications. In: Advances in Cryptology – CRYPTO 2016, vol. 9816, pp. 93–122. Springer Berlin Heidelberg, Berlin, Heidelberg (2016). https://doi.org/10.1007/978-3-662-53015-3_4
24. Durak, F.B., Weng, C., Anderson, E., Laine, K., Chase, M.: Precio: Private Aggregate Measurement via Oblivious Shuffling. In: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (2024)
25. Dwork, C.: Differential Privacy. In: Bugliesi, M., Preneel, B., Sassone, V., Wegener, I. (eds.) Automata, Languages and Programming. pp. 1–12. Lecture Notes in Computer Science, Springer, Berlin, Heidelberg (2006). https://doi.org/10.1007/11787006_1
26. Dwork, C., Roth, A.: The Algorithmic Foundations of Differential Privacy. Foundations and Trends® in Theoretical Computer Science **9**(3–4), 211–407 (Aug 2014). <https://doi.org/10.1561/04000000042>, <https://www.nowpublishers.com/article/Details/TCS-042>, publisher: Now Publishers, Inc.
27. Eskandarian, S., Corrigan-Gibbs, H., Zaharia, M., Boneh, D.: Express: Lowering the cost of metadata-hiding communication with cryptographic privacy. In: 30th USENIX Security Symposium (USENIX Security 21). pp. 1775–1792 (2021)
28. Gilboa, N., Ishai, Y.: Distributed Point Functions and Their Applications. In: Nguyen, P.Q., Oswald, E. (eds.) Advances in Cryptology – EUROCRYPT 2014. pp. 640–658. Lecture Notes in Computer Science, Springer, Berlin, Heidelberg (2014). https://doi.org/10.1007/978-3-642-55220-5_35
29. Goel, A., Wang, M., Wang, Z.: Multiparty Distributed Point Functions (2025), <https://eprint.iacr.org/2025/1074>, publication info: Published by the IACR in CRYPTO 2025
30. Hadar, J., Russell, W.R.: Rules for ordering uncertain prospects. The American economic review **59**(1), 25–34 (1969), publisher: JSTOR
31. Kairouz, P., Liu, Z., Steinke, T.: The Distributed Discrete Gaussian Mechanism for Federated Learning with Secure Aggregation. In: Proceedings of the 38th International Conference on Machine Learning. pp. 5201–5212. PMLR (Jul 2021), <https://proceedings.mlr.press/v139/kairouz21a.html>, iSSN: 2640-3498
32. Kumar, C., Patranabis, S., Mukhopadhyay, D.: Compact Key Function Secret Sharing with Non-linear Decoder. IACR Communications in Cryptology **1**(2) (Jul 2024). <https://doi.org/10.62056/a3c3c3w9p>, <https://cic.iacr.org/p/1/2/8>
33. Mohamad, M., Jaballah, W.B., Önen, M., Conti, M.: SoK: Secure Aggregation based on cryptographic schemes for Federated Learning. Proceedings on Privacy Enhancing Technologies p. 17 (2023)
34. Newman, Z., Servan-Schreiber, S., Devadas, S.: Spectrum: High-bandwidth Anonymous Broadcast. In: 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22). pp. 229–248. USENIX Association, Renton, WA (2022), <https://www.usenix.org/conference/nsdi22/presentation/newman>

35. Ogilvie, T.: Differential Privacy for Free? Harnessing the Noise in Approximate Homomorphic Encryption. In: Oswald, E. (ed.) *Topics in Cryptology – CT-RSA 2024*. pp. 292–315. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-58868-6_12
36. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM* **56**(6), 34:1–34:40 (2009). <https://doi.org/10.1145/1568318.1568324>
37. Ryffel, T., Tholoniati, P., Pointcheval, D., Bach, F.: AriaNN: Low-Interaction Privacy-Preserving Deep Learning via Function Secret Sharing. *Proceedings on Privacy Enhancing Technologies* **1**, 291–316 (2022)
38. Servan-Schreiber, S., Beyzerov, S., Yablon, E., Park, H.: Private Access Control for Function Secret Sharing. In: *2023 IEEE Symposium on Security and Privacy (SP)*. pp. 809–828 (2023). <https://doi.org/10.1109/SP46215.2023.10179295>
39. Shaked, M., Shanthikumar, J.G.: *Univariate Stochastic Orders*. In: *Stochastic Orders*, pp. 3–79. Springer, New York, NY (2007). https://doi.org/10.1007/978-0-387-34675-5_1
40. Shi, S., Chu, X., Cheung, K.C., See, S.: Understanding Top-k Sparsification in Distributed Deep Learning (2019). <https://doi.org/10.48550/arXiv.1911.08772>, <http://arxiv.org/abs/1911.08772>
41. Smith, A., Ullman, J.: *Privacy in Statistics and Machine Learning: Approximate Differential Privacy* (2021), lecture notes
42. Stevens, T., Skalka, C., Vincent, C., Ring, J., Clark, S., Near, J.: Efficient Differentially Private Secure Aggregation for Federated Learning via Hardness of Learning with Errors. In: *31st USENIX Security Symposium (USENIX Security 22)* (2022), <https://www.usenix.org/conference/usenixsecurity22/presentation/stevens>
43. Sébert, A.G., Zuber, M., Stan, O., Sirdey, R., Gouy-Pailler, C.: When approximate design for fast homomorphic computation provides differential privacy guarantees (Apr 2023), <http://arxiv.org/abs/2304.02959>, arXiv:2304.02959 [cs]
44. Talwar, K.: Differential Secrecy for Distributed Data and Applications to Robust Differentially Secure Vector Summation. In: *DROPS-IDN/v2/document/10.4230/LIPIcs.FORC.2022.7*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2022). <https://doi.org/10.4230/LIPIcs.FORC.2022.7>
45. Vadapalli, A.: *Distributed Point Functions and Their Applications to Secure Multi-Party Computation*. Ph.D. thesis, Indiana University (2021)
46. Vadapalli, A., Storrier, K., Henry, R.: Sabre: Sender-Anonymous Messaging with Fast Audits. In: *2022 IEEE Symposium on Security and Privacy (SP)*. pp. 1953–1970 (2022). <https://doi.org/10.1109/SP46214.2022.9833601>
47. Wagh, S.: Pika: Secure Computation using Function Secret Sharing over Rings. *Proceedings on Privacy Enhancing Technologies* p. 27 (2022)
48. Wang, F., Yun, C., Goldwasser, S., Vaikuntanathan, V., Zaharia, M.: Splinter: Practical Private Queries on Public Data. In: *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. pp. 299–313 (2017)
49. Zalonis, J., Armknecht, F., Scheu-Hachtel, L.: Differentially Private Functional Encryption. *Proceedings on Privacy Enhancing Technologies* (2024), <https://petsymposium.org/popets/2024/popets-2024-0061.php>
50. Zyskind, G., Yanai, A., Pentland, A.S.: High-Throughput Three-Party DPFs with Applications to ORAM and Digital Currencies. In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security* (2024)

A Proof of Theorem 1

Proof. First, observe that: $\mathcal{D}_{f_1, x^{(1)}, \dots, f_m, x^{(m)}}^\lambda - \sum_{j=1}^m f_j(x^{(j)})$

$$\begin{aligned}
&= \text{Decode} \left(\sum_{j=1}^m \text{Eval}(k_1^{(j)}, x^{(j)}), \dots, \sum_{j=1}^m \text{Eval}(k_p^{(j)}, x^{(j)}) \right) \\
&\quad - \sum_{j=1}^m f_j(x^{(j)}) \\
&= \sum_{i=0}^p \left(\sum_{j=1}^m \text{Eval}(k_i^{(j)}, x^{(j)}) \right) - \sum_{j=1}^m f_j(x^{(j)}) \\
&= \sum_{j=1}^m \left(\sum_{i=0}^p (\text{Eval}(k_i^{(j)}, x^{(j)}) - f_j(x^{(j)})) \right)
\end{aligned}$$

Reusing the notation from Definition 8:

$$\begin{aligned}
\mathcal{D}_{f_j, x^{(j)}}^\lambda &:= \sum_{i=1}^p \text{Eval}(k_i^{(j)}, x^{(j)}) \\
&\Rightarrow \mathcal{D}_{f_1, x^{(1)}, \dots, f_m, x^{(m)}}^\lambda - \sum_{j=1}^m f_j(x^{(j)}) \\
&= \sum_{j=1}^m \left(\mathcal{D}_{f_j, x^{(j)}}^\lambda - f_j(x^{(j)}) \right)
\end{aligned}$$

For convenience, we define the random variable $\mathcal{D}_j^- = \left(\mathcal{D}_{f_j, x^{(j)}}^\lambda - f_j(x^{(j)}) \right)$.

Thus, we want to prove that $\left(\sum_{j=1}^m \mathcal{D}_j^- \right)^2 \preceq (\psi_*)^2$.

Note that we have:

$$\begin{aligned}
&\text{For any sequence } \{d_1 \dots, d_m\}, \\
&\left(\sum_{j=1}^m d_j \right)^2 \leq \left(m \cdot \max_j |d_j| \right)^2 = m^2 \max_j \left(d_j \right)^2 \\
&\Rightarrow \left\{ \left(\sum_{j=1}^m \mathcal{D}_j^- \right)^2 \geq t \right\} \subseteq \left\{ \max_j \left(\mathcal{D}_j^- \right)^2 \geq t/m^2 \right\} \\
&\quad \subseteq \bigcup_{j=1}^m \left\{ \left(\mathcal{D}_j^- \right)^2 \geq t/m^2 \right\}
\end{aligned}$$

By the union bound:

$$\Pr \left[\left(\sum_{j=1}^m \mathcal{D}_j^- \right)^2 \geq t \right] \leq \sum_{j=1}^m \Pr \left[\left(\mathcal{D}_j^- \right)^2 \geq t/m^2 \right] \quad (1)$$

By assumption, the scheme is ψ -noisy

$$\Rightarrow \forall j, \left(\mathcal{D}_j^- \right)^2 \preceq (\psi)^2 \quad (2)$$

$$\iff \forall j, \forall t, \Pr \left[\left(\mathcal{D}_j^- \right)^2 \geq t \right] \leq \Pr[(\psi)^2 \geq t] \quad (3)$$

By combining Equations (1) and (3):

$$\Pr \left[\left(\sum_{j=1}^m \mathcal{D}_j^- \right)^2 \geq t \right] \leq m \Pr[(\psi)^2 \geq t/m^2] \quad (4)$$

By definition, ψ_* is such that $\Pr[\psi_*^2 \geq t] = \min\{1, m \Pr[\psi^2 \geq t/m^2]\}, \forall t$. By injecting this into Equation (4), we obtain:

$$\begin{aligned} \Pr \left[\left(\sum_{j=1}^m \mathcal{D}_j^- \right)^2 \geq t \right] &\leq \Pr[\psi_*^2 \geq t] \\ \iff \left(\sum_{j=1}^m \mathcal{D}_j^- \right)^2 &\preceq (\psi_*)^2 \end{aligned}$$

The scheme $\mathcal{E}$ is then ψ_* -noisy linear compressible.

B Proof of Theorem 3

Proof. Given the key $k_i = (b_i, s_i, v)$ of party i and a point $x \in \mathcal{X}$, the corresponding share is

$$\text{Eval}((b_i, s_i, v), x) = g_i[x_r] + b_i[x_l]v[x_r] + \text{err}_i$$

where $g_i \leftarrow G_{\text{LWE}}(s_i[x_l])$, $\text{err}_i \leftarrow \mathcal{N}(0, \sigma_{\text{flood}}^2)$ and (x_l, x_r) is the representation of x in the $\nu \times \nu$ grid. Direct calculation shows that

$$\begin{aligned} \mathcal{D}_{f,x} &= \text{Decode}(\text{Eval}(k_1, x), \dots, \text{Eval}(k_p, x)) \\ &= \sum_{i=1}^p \left(G_{\text{LWE}}(s_i[x_l])[x_r] + b_i[x_l]v[x_r] + \text{err}_i \right) \\ &= \left(\sum_{i=1}^p G_{\text{LWE}}(s_i[x_l]) \right) [x_r] + v[x_r] \sum_{i=1}^p b_i[x_l] + \sum_{i=1}^p \text{err}_i \\ &= G_{\text{LWE}}(s_* \cdot e_{\alpha_l}[x_l])[x_r] + v[x_r] \cdot e_{\alpha_l}[x_l] \\ &\quad + \sum_{i=1}^p \text{err}_i + \sum_{i=1}^p \text{err}'_i \end{aligned}$$

where $\text{err}'_i \leftarrow \mathcal{N}(0, \sigma_{\text{LWE}}^2)$ captures the noise from $G_{\text{LWE}}(s_i[x_l])$. Now, by cases:

- If $x_l = \alpha_l$, namely we are in the row encoding α , recalling that $v = \beta e_{\alpha_r} - G_{\text{LWE}}(s_*)$, we have

$$\begin{aligned} \mathcal{D}_{f,x} &= G_{\text{LWE}}(s_*)[x_r] + \beta e_{\alpha_r}[x_r] - G_{\text{LWE}}(s_*)[x_r] \\ &\quad + \sum_{i=1}^p \text{err}_i + \sum_{i=1}^p \text{err}'_i \\ &= \beta e_{\alpha_r}[x_r] + \sum_{i=1}^p \text{err}_i + \sum_{i=1}^p \text{err}'_i + \text{err}'_* , \\ &\quad \text{with } \text{err}'_* \leftarrow \mathcal{N}(0, \sigma_{\text{LWE}}^2) . \end{aligned}$$

- Else, we have

$$\mathcal{D}_{f,x} = \sum_{i=1}^p \text{err}_i + \sum_{i=1}^p \text{err}'_i .$$

So we have that $\mathcal{D}_{f,x} - f(x) = \sum_{i=1}^p \text{err}_i + \sum_{i=1}^p \text{err}'_i + \text{err}'_* \cdot e_{\alpha_l}[x_l]$, which behaves as $\mathcal{N}(0, (p+1)\sigma_{\text{LWE}}^2 + p\sigma_{\text{flood}}^2)$ when $x_l = \alpha_l$ and as $\mathcal{N}(0, p\sigma_{\text{LWE}}^2 + p\sigma_{\text{flood}}^2)$ otherwise. So, $\mathcal{D}_{f,x} - f(x)$ is stochastically dominated in absolute value by the distribution with largest variance, that is $\psi \sim \mathcal{N}(0, (p+1)\sigma_{\text{LWE}}^2 + p\sigma_{\text{flood}}^2)$, hence the scheme is ψ -noisy.

In the description and proofs, we chose to use the (continuous) Gaussian distribution to simplify the analysis and keep it more readable. In practice, since computations are carried out over finite fields, a discrete version should be used instead. The proofs would carry over with minimal changes, with only some care needed since the sum of two discrete Gaussians is not itself a discrete Gaussian, existing works [15, 31] provided bounds on the approximation error.

Observe that the approximation error induced by the use of continuous Gaussians is really small [31]. For example, for two discrete Gaussians $X \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$ and $Y \sim \mathcal{N}_{\mathbb{Z}}(0, \tau^2)$, the Rényi divergence between $X+Y$ and $W \sim \mathcal{N}_{\mathbb{Z}}(0, \tau^2 + \sigma^2)$ is only 10^{-12} when $\sigma^2 = \tau^2 = 3$ [31].

C Proof of Theorem 4

This appendix proves the FPO property for our DPF scheme, but first we recall some results on statistical indistinguishability.

Lemma 1. *Let A, B be two centered Gaussian random variables $A \sim \mathcal{N}(0, \sigma_A^2)$ and $B \sim \mathcal{N}(0, \sigma_B^2)$. The advantage of any binary distinguisher $\mathcal{D}$ (not necessarily polynomially-bounded) is bounded by:*

$$|\Pr[\mathcal{D}(A) = 1] - \Pr[\mathcal{D}(B) = 1]| \leq \frac{1}{2} \sqrt{\frac{\sigma_A^2}{\sigma_B^2} - \ln \frac{\sigma_A^2}{\sigma_B^2} - 1} .$$

Proof. For any two random variables A, B , we have that

$$|\Pr[\mathcal{D}(A) = 1] - \Pr[\mathcal{D}(B) = 1]| \leq \Delta(A, B)$$

where $\Delta(A, B)$ is the statistical distance between A and B , defined as

$$\Delta(A, B) := \frac{1}{2} \sum_x |\Pr[A = x] - \Pr[B = x]|.$$

By the Pinsker's inequality we know that

$$\Delta(A, B) \leq \sqrt{\frac{1}{2} D_{\text{KL}}(A \parallel B)}$$

where $D_{\text{KL}}(A \parallel B)$ is the Kullback–Leibler (KL) divergence of A and B , defined as

$$D_{\text{KL}}(A \parallel B) := \sum_x \Pr[A = x] \cdot \ln \frac{\Pr[A = x]}{\Pr[B = x]}.$$

For $A \sim \mathcal{N}(0, \sigma_A^2)$ and $B \sim \mathcal{N}(0, \sigma_B^2)$ direct calculations show that their KL divergence is

$$D_{\text{KL}}(A \parallel B) = \frac{1}{2} \left(\frac{\sigma_A^2}{\sigma_B^2} - \ln \frac{\sigma_A^2}{\sigma_B^2} - 1 \right).$$

A distinguisher advantage is then upper-bounded by

$$|\Pr[\mathcal{D}(A) = 1] - \Pr[\mathcal{D}(B) = 1]| \leq \frac{1}{2} \sqrt{\frac{\sigma_A^2}{\sigma_B^2} - \ln \frac{\sigma_A^2}{\sigma_B^2} - 1}.$$

Using this result, we can prove Theorem 4:

Proof. Without loss of generality, let us assume that the corrupted subset of parties is $A = \{1, \dots, t\}$. We build a simulator $\mathcal{S}$ for Definition 9

$$\mathcal{S} = \mathcal{S}(\lambda, \mathcal{F}, \{(x_1, y_1), \dots, (x_n, y_n)\})$$

as follows. The simulator samples random quantities $\tilde{b}_1, \dots, \tilde{b}_t \in \mathbb{Z}_q^\nu$, $\tilde{s}_1, \dots, \tilde{s}_t \in (\mathbb{Z}_q^m)^\nu$, and $v \in \mathbb{Z}_q^\nu$. The shares of the corrupted parties can be computed following the protocol: $\tilde{y}_{i,j} = G_{\text{LWE}}(\tilde{s}_i[x_l^{(j)}])[x_r^{(j)}] + \tilde{b}_i[x_l^{(j)}]\tilde{v}[x_r^{(j)}] + \text{err}_{i,j}$ where $\text{err}_{i,j} \sim \mathcal{N}(0, \sigma_{\text{flood}}^2)$. The shares of the honest parties, which still belong to the view of the adversary, are all sampled uniformly at random, but the last share, which is computed as

$$\tilde{y}_{p,j} = -(\tilde{y}_{1,j} + \dots + \tilde{y}_{p-1,j}) + y_j + \sum_{i=1}^p \text{ERR}_{i,j}$$

where $\text{ERR}_{i,j}$ is the sum of the previously sampled flooding and LWE errors for $i \leq t$, while it is freshly sampled from $\mathcal{N}(0, \sigma_{\text{LWE}}^2 + \sigma_{\text{flood}}^2)$ for $i > t$. The simulated view is then

$$(\{\tilde{b}_i, \tilde{s}_i, \tilde{v}\}_{i \in A}, \{\tilde{y}_{1,1}, \dots, \tilde{y}_{p,n}\}, \{(x_1, y_1), \dots, (x_n, y_n)\}).$$

Note that even if we do not mention it explicitly (for the sake of having lighter notation), all randomness of the corrupted parties is also part of the adversary's view. The keys $(\tilde{b}_i, \tilde{s}_i, \tilde{v})$ are clearly indistinguishable for the same reason as in the proof of Theorem 2. For any $j \in \{1, \dots, n\}$, each share $\tilde{y}_{i,j}$ is individually indistinguishable from its real counterpart due to the first term $G_{\text{LWE}}(\tilde{s}_i[x_l^{(j)}])[x_r^{(j)}]$ acting as a uniform mask. Moreover, it can be shown that no subset of these shares provide any useful information to the distinguisher, but when all shares for a specific x_j are considered, which is the only interesting case. In this case, their sum leads to

$$\tilde{y}_{1,j} + \dots + \tilde{y}_{p,j} = y_j + \text{ERR}_j$$

with $\text{ERR}_j := \sum_{i=1}^p \text{ERR}_{i,j} \sim \mathcal{N}(0, p(\sigma_{\text{LWE}}^2 + \sigma_{\text{flood}}^2))$. As we have seen in the proof of Theorem 3, the decoding of the real shares leads to two slightly different error distributions depending on whether we are on the row encoding α or not. In particular, we either get a variance $\sigma_A^2 = (p+1)\sigma_{\text{LWE}}^2 + p\sigma_{\text{flood}}^2$ or $\sigma_B^2 = p\sigma_{\text{LWE}}^2 + p\sigma_{\text{flood}}^2$.

The distinguisher can potentially remove the randomness of the dishonest parties to make its job easier. We hence consider the worst case of

$$\begin{aligned} \text{ERR}_j &\sim \mathcal{N}(0, (p-t)(\sigma_{\text{LWE}}^2 + \sigma_{\text{flood}}^2)) \\ \sigma_A^2 &= (p-t+1)\sigma_{\text{LWE}}^2 + (p-t)\sigma_{\text{flood}}^2 \\ \sigma_B^2 &= (p-t)\sigma_{\text{LWE}}^2 + (p-t)\sigma_{\text{flood}}^2 \end{aligned}$$

and show that if $\sigma_{\text{flood}} \geq \sqrt{2^{\lambda-1.5}/(p-t)-1} \sigma_{\text{LWE}}$, it is not possible to distinguish the simulated shares' decoding in the first case σ_A better than $\leq 2^{-\lambda}$ (the second case σ_B is trivial as the error distribution coincides with the one of err_j). Let $x = \sigma_A^2/\sigma_B^2$. Thanks to Lemma 1, we just need to prove that $1/2\sqrt{x - \ln x - 1} \leq 2^{-\lambda}$. Since $\sigma_A^2 > \sigma_B^2$ we have that $x > 1$. By taking a Taylor expansion of order 1 of $x - \ln x - 1$, we get $(x-1)^2/2$, in particular we have that $x - \ln x - 1 < (x-1)^2/2$ for all $x > 1$. Hence,

$$\frac{1}{2}\sqrt{x - \ln x - 1} < \frac{1}{2}\sqrt{(x-1)^2/2} = \frac{x-1}{2\sqrt{2}}.$$

Now,

$$\begin{aligned} x &= \frac{\sigma_A^2}{\sigma_B^2} = \frac{(p-t+1)\sigma_{\text{LWE}}^2 + (p-t)\sigma_{\text{flood}}^2}{(p-t)\sigma_{\text{LWE}}^2 + (p-t)\sigma_{\text{flood}}^2} \\ &= \frac{\sigma_{\text{LWE}}^2}{(p-t)\sigma_{\text{LWE}}^2 + (p-t)\sigma_{\text{flood}}^2} + 1. \end{aligned}$$

So,

$$\begin{aligned} \frac{x-1}{2\sqrt{2}} \leq 2^{-\lambda} &\Leftrightarrow \frac{\sigma_{\text{LWE}}^2}{(p-t)(\sigma_{\text{LWE}}^2 + \sigma_{\text{flood}}^2)} \leq 2^{1.5-\lambda} \\ &\Leftrightarrow \sigma_{\text{flood}}^2 \geq \left(\frac{2^{\lambda-1.5}}{p-t} - 1 \right) \sigma_{\text{LWE}}^2. \end{aligned}$$

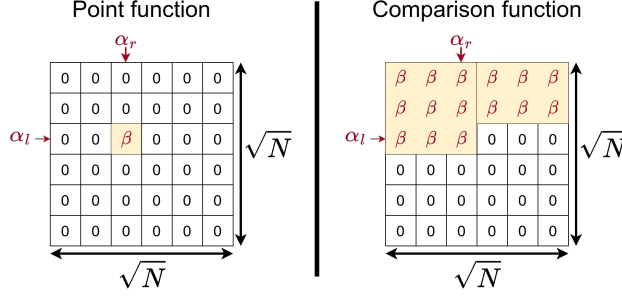


Fig. 2. Comparison of the grid representation of point and comparison functions.

D Noisy DCF scheme

This section adapts our noisy DPF to build a noisy DCF. DCF schemes enable to share comparison functions f such that $f(x) = \beta$ for any $x \geq \alpha$, 0 otherwise.

D.1 Scheme

To adapt our DPF, we take inspiration from Boyle et al. [10] who designed a DCF based on their DPF. To understand this adaption, consider the function domain as a $\sqrt{N} \times \sqrt{N}$ grid. While the grid of point functions is full of zeros except on one cell containing β , the grid of comparison functions has three types of row (in this specific order): (1) $\alpha_l - 1$ rows containing only the value β , (2) a unique row containing α_r times the value β at the beginning and only zeros at the end, and (3) rows containing only the value 0. In other words, there is a row before which all rows contain β and after which all rows contain 0. Figure 2 illustrates this phenomenon.

To build our noisy DCF, we propose to add a secret-shared vector w containing β on the first $\alpha_l - 1$ position and 0 otherwise. This vector basically stores the β value for all the rows containing only β . Then, we update the vector v (already present in our noisy DPF) so that the masked vector contains α_r times the value β . We detail this adaptation in Algorithm 3 and highlight the changes compared to our noisy DPF in blue.

D.2 Security and share properties

Theorem 5 (Privacy). *Let $\lambda \in \mathbb{N}$, $N, p \in \mathbb{N}$. For any distribution χ , the scheme $\mathcal{E} = (\text{Gen}, \text{Eval}_\chi, \text{Decode})$ as described in Algorithm 3 is an FSS scheme for the family of all comparison functions with $\alpha \in \{1, \dots, N\}$ and $\beta \in \mathbb{Z}_q$.*

Assuming that the LWE assumption holds, then the scheme $\mathcal{E}$ is private against at most t semi-honest parties with $t < p$.

Algorithm 3 Dishonest-majority noisy DCF scheme (differences with our noisy DPF of Algorithm 2 highlighted in blue)

- 1: Let $\nu \leftarrow \lceil \sqrt{N} \rceil$
 - 2: Represent the function domain $\{1 \dots N\}$ as a $(\sqrt{N} \times \sqrt{N})$ grid
 - 3: Let $G_{LWE} : \mathbb{Z}_q^m \rightarrow \mathbb{Z}_q^\nu$ be an LWE-based PRG using the Gaussian distribution $\mathcal{N}(0, \sigma_{LWE}^2)$ as error distribution. $\triangleright m = O(\lambda)$
 - 4: **function** Gen($\lambda, (\alpha, \beta), p$)
 - 5: Parse α as (α_l, α_r) (i.e., representation in the grid)
 - 6: Sample random vectors $b_1, \dots, b_{p-1}$ from $(\mathbb{Z}_q)^\nu$
 - 7: Sample random vectors $w_1, \dots, w_{p-1}$ from $(\mathbb{Z}_q)^\nu$
 - 8: Sample vectors of seeds $s_1, \dots, s_{p-1}$ from $(\mathbb{Z}_q^m)^\nu$
 - 9: Sample a single seed s_* from $\mathbb{Z}_q^m$
 - 10: Set $b_p \in (\mathbb{Z}_q)^\nu$ such that $\sum_{i=1}^p b_i = e_{\alpha_l}$
 $\triangleright e_x$ is a vector equal to 1 on index x and 0 otherwise
 - 11: Set $s_p \in (\mathbb{Z}_q^m)^\nu$ such that $\sum_{i=1}^p s_i = s_* \cdot e_{\alpha_l}$
 - 12: Set $w_p \in (\mathbb{Z}_q)^\nu$ such that $\sum_{i=1}^p w_i = \beta \cdot c_{\alpha_l-1}$
 $\triangleright c_x$ is a vector equal to 1 on any index $i \leq x$ and 0 otherwise
 - 13: Let $v \leftarrow \beta \cdot c_{\alpha_r} - G_{LWE}(s_*)$
 - 14: Build the DPF keys $k_1, \dots, k_p$ as follows: $k_i = (b_i, s_i, v, w_i)$
 - 15: **return** $(k_1, \dots, k_p)$
 - 16: **function** Eval $_\chi(k_i, x)$
 - 17: Parse k_i as (b_i, s_i, v, w_i) and x as (x_l, x_r) (i.e., representation in the $\nu \times \nu$ grid)
 - 18: Expand the x_l -th random seed: $g_i \leftarrow G_{LWE}(s_i[x_l])$
 - 19: Let $y_i \leftarrow w_i[x_l] + g_i[x_r] + b_i[x_l]v[x_r]$
 - 20: Sample an error $\text{err} \in \mathbb{Z}_q$ using the distribution χ
 - 21: Let $y_i \leftarrow y_i + \text{err}$ $\triangleright$ To have a function-private output
 - 22: **return** y_i
 - 23: **function** Decode($y_1, \dots, y_p$)
 - 24: **return** $\sum_{i=1}^p y_i$
-

Proof. We can trivially adapt the proof of Theorem 2. The simulator Sim returns the following: $\tilde{b}_i \xleftarrow{R} (\mathbb{Z}_q)^\nu, \tilde{s}_i \xleftarrow{R} \mathbb{Z}_q^m, \tilde{v} \xleftarrow{R} \mathbb{Z}_q^\nu, \tilde{w}_i \xleftarrow{R} (\mathbb{Z}_q)^\nu$.

For the same reasons already mentioned in the proof of Theorem 2, the simulation of b_i, s_i, v is indistinguishable from the output of Gen. Finally, like b_i and s_i , the vector w_i outputted by Gen is random, so the simulator's output distribution $\tilde{w}_i$ is computationally indistinguishable from that induced by the distribution of a single output of Gen.

Like our noisy DPF, the privacy of our noisy DPF scheme holds for any distribution χ used in the Eval $_\chi$; including δ_0 , the Dirac distribution on 0 (i.e., the evaluator adds no noise).

Theorem 6 (Correctness). *Let $\mathcal{E} = (\text{Gen}, \text{Eval}_\chi, \text{Decode})$ be the DCF scheme described in Algorithm 3, with $\chi \sim \mathcal{N}(0, \sigma_{\text{flood}}^2)$. Let the noise used for the PRG*

Table 2. Comparison of the multi-party DCF schemes.Notations: p parties, N the domain size, output in $\mathbb{Z}_q$.

Scheme	Hardness	Compress.	FPO	Key size
Dishonest-majority schemes				
Trivial	None	✓	✓	$N \cdot \log q$
[10]	PRG	✓	✓	$O(\sqrt{N} \cdot q^{\frac{p-1}{2}} \cdot (\log q + \lambda))$
[32]	DDH	✗	✗	$O(\sqrt{N} \cdot (\lambda + \log q))$
Ours	LWE	✓(noisy)	✓	$O(\sqrt{N} \cdot \sqrt{(1 + \lambda)} \cdot \log q)$
Honest-majority schemes				
None				

and for flooding have Gaussian distribution with standard deviation σ_{LWE} and σ_{flood}^2 , respectively. Let p be the number of parties. Then $\mathcal{E}$ is ψ -noisy correct with $\psi \sim \mathcal{N}(0, (p+1)\sigma_{LWE}^2 + p\sigma_{\text{flood}}^2)$.

Proof. Follows the same structure as the proof of Theorem 3.

Share properties As our noisy DCF is a straightforward adaptation of our noisy DPF, the noisy DCF has the same properties as the noisy DPF: 0-succinctness, function-private output, and noisy compressibility. As the theorems and proofs are straightforward adaptations of those presented in Subsection 5.4, we do not re-develop them here.

D.3 Comparison with state-of-the-art

For multi-party DCF, Boyle et al. [10] presented a PRG-based scheme (derived from their DPF) and Kumar et al. [32] presented an elliptic-curve-based scheme (derived from the DPF of [17]). Both schemes are secure against a dishonest majority and have $O(\sqrt{N})$ key size.

Table 2 compares our noisy DCF to existing DCF schemes, based on their key size and share properties. Our scheme is the only (non-trivial) scheme providing both linear compressibility and a key size without an exponential factor. On the one hand, Boyle et al. [10] have an exponential factor q^p in their key size. On the other hand, Kumar et al. [32] provides neither compressibility nor function private output.

No existing work describes an honest-majority DCF. As observed in Algorithm 3, we can usually adapt DPF schemes to DCF. However, this adaptation of existing DPF is not in the scope of our work. Table 2 is sufficient to show that noisy DCF can improve the state-of-the-art in dishonest-majority schemes.